porting_remaining_agents

Comprehensive Porting Plan: ALL REMAINING CLI Agents into HelixCode

Mission: Port 27 remaining CLI agents from helix_agent/cli_agents into HelixCode
Date: 2026-05-04 **HelixCode Module**: `dev.helix.code` **Agents Covered**: 27
Total Features Identified: 80+

Table of Contents

---

1. Executive Summary

1.1 Already Ported Agents (10)

Agent	Status	Document
Aider	DONE	`porting_aider.md`
Claude Code	DONE	`porting_claude_code.md`
Cline	DONE	`porting_cline.md`
Codex	DONE	`porting_codex.md`
Continue.dev	DONE	`porting_continue_dev.md`
Forge	DONE	`porting_forge.md`
Kilo Code	DONE	`porting_kilo_code.md`
OpenCode	DONE	`porting_opencode.md`
OpenHands	DONE	`porting_openhands.md`
Plandex	DONE	`porting_plandex.md`

1.2 Remaining Agents to Port (27)

The following agents have been analyzed and their unique features identified for porting into HelixCode:

#	Agent	Language	Tier	Priority
1	**Claude-Code-Plugins-And-Skills**	TypeScript	1	P0
2	**Bridle**	TypeScript	2	P1
3	**Codai**	Python	3	P1
4	**Codename_Goose**	Rust	1	P0
5	**Conduit**	TypeScript	4	P2
6	**DeepSeek_CLI**	Python	3	P2
7	**Emdash**	TypeScript/Electron	1	P0
8	**FauxPilot**	Python	4	P2
9	**Gemini_CLI**	TypeScript	1	P0
10	**Get-Shit-Done**	Unknown	5	P2
11	**GitHub-Copilot-CLI**	TypeScript	1	P0
12	**GitHub-Spec-Kit**	Ruby	2	P1
13	**GitMCP**	TypeScript	4	P1
14	**Mistral_Code**	TypeScript	1	P0
15	**MobileAgent**	Python	4	P2
16	**Multiagent-Coding-System**	TypeScript	4	P1
17	**Nanocoder**	TypeScript	3	P1
18	**Noi**	TypeScript/Electron	4	P2
19	**Octogen**	Python	4	P2
20	**Ollama_Code**	TypeScript	3	P2
21	**Postgres-MCP**	TypeScript	4	P1
22	**Qwen_Code**	TypeScript	1	P0
23	**Shai**	Rust	3	P1
24	**SnowCLI**	Python	4	P2
25	**Stark-Kitty-Kiro-Cli**	TypeScript	3	P2
26	**Superset**	Python	4	P2
27	**TaskWeaver**	Python	2	P0
28	**Warp**	Rust	1	P0
29	**vtcode**	Swift	3	P1
30	**gptme**	Python	2	P1

---

2. Already Ported (10 agents) - Reference

For completeness, the following agents have already been ported. Their documents exist in `/mnt/agents/output/`:

- `porting_aider.md` - Repository mapping, edit blocks, voice commands
- `porting_claude_code.md` - Multi-agent swarms, XML communication, YOLO classifier
- `porting_cline.md` - Browser automation, computer use, screenshot capture
- `porting_codex.md` - Sandboxed execution, seatbelt integration, rust core
- `porting_continue_dev.md` - Context providers, autocomplete, chat interface
- `porting_forge.md` - Domain-specific languages, workflow engine, action chains
- `porting_kilo_code.md` - Agent manager, diff panel, PR import, code review
- `porting_opencode.md` - TUI dashboard, Bubble Tea interface, LSP integration
- `porting_openhands.md` - Browser automation, code act, runtime server
- `porting_plandex.md` - Task decomposition, planning tree, verification

---

3. Agent Porting Plans (27 agents)

Agent 1: Claude-Code-Plugins-And-Skills

> **Source**: `cli_agents/claude-plugins`, `cli_agents/claude-squad`, `cli_agents/codex-skills` **Language**: TypeScript **License**: Mixed (Anthropic proprietary + MIT skills)

Feature Inventory - Top 3 Unique Features

1. SKILL.md System - Progressive Disclosure Skill Architecture - Skills are directories with `SKILL.md` containing YAML frontmatter + instructions - Progressive disclosure: metadata loaded first, full instructions only when skill is selected - Open Agent Skills standard - portable across Claude Code, Codex, Gemini CLI, Cursor - Skill levels: Enterprise > Personal > Project > Plugin - Live change detection - skills hot-reload without session restart

2. Plugin Marketplace Ecosystem - `/plugin marketplace add <repo>` - add curated skill repositories - `/plugin install <skill>@<source>` - install by domain or individual skill - Plugin bundling: skills + agents + commands + MCPs in single package - Namespace system: `plugin-name:skill-name` prevents conflicts - 232+ community skills available

3. Agent Teams - Multi-Agent Coordination - Lead agent + teammates with independent context windows - Shared task list (markdown file) for coordination - Mailbox messaging system for peer-to-peer communication - Unlike subagents, teammates message each other directly - Experimental but powerful for parallel research/review/debugging

HelixCode Integration

Feature	HelixCode Package	Integration Point
SKILL.md system	`internal/skills/`	New package - skill registry, progressive disclosure

Feature	HelixCode Package	Integration Point
Plugin marketplace	`cmd/helix plugin/`	New subcommand - marketplace management
Agent teams	`internal/agent/swarm/`	Extend existing swarm with shared task list + mailbox

Exact Code Changes

New File: `internal/skills/registry.go`

```
package skills

import (
    "context"
    "os"
    "path/filepath"
    "strings"
    "sync"
    "time"
)

// Skill represents a loaded SKILL.md with progressive disclosure
type Skill struct {
    Name         string
    Description string
    Path         string
    Metadata     SkillMetadata
    Instructions string      // loaded on demand
    Scripts      []string
    References   []string
    Level        SkillLevel // enterprise, personal, project,
        plugin
}

type SkillMetadata struct {
    Name            string   `yaml:"name"`
    Description     string   `yaml:"description"`
    AllowedTools    []string `yaml:"allowed-tools"`
    DisableModelInvocation bool `yaml:"disable-model-invocation"`
    Context         string   `yaml:"context"`
    UserInvocable   bool     `yaml:"user-invocable"`
}

type SkillLevel int

const (
    SkillLevelEnterprise SkillLevel = iota
    SkillLevelPersonal
```

```
    SkillLevelProject
    SkillLevelPlugin
)

// Registry manages skill discovery and loading
type Registry struct {
    mu      sync.RWMutex
    skills  map[string]*Skill
    watchers map[string]*fsnotify.Watcher

    // Search paths in precedence order
    paths []string
}

func NewRegistry() *Registry {
    return &Registry{
        skills:   make(map[string]*Skill),
        watchers: make(map[string]*fsnotify.Watcher),
        paths: []string{
            // Enterprise (managed settings)
            "/etc/helix/skills/",
            // Personal
            filepath.Join(os.Getenv("HOME"), ".helix/skills/"),
            // Project
            ".helix/skills/",
        },
    }
}

// Discover scans all skill paths and loads metadata (not full
        instructions)
func (r *Registry) Discover(ctx context.Context) error {
    for _, path := range r.paths {
        if _, err := os.Stat(path); os.IsNotExist(err) {
            continue
        }
        // Scan for SKILL.md files
        err := filepath.Walk(path, func(walkPath string, info
        os.FileInfo, err error) error {
            if err != nil || info.IsDir() {
                return err
            }
            if strings.ToLower(info.Name()) == "skill.md" {
                skillDir := filepath.Dir(walkPath)
                skill, err := r.loadSkillMetadata(skillDir)
                if err == nil {
                    r.mu.Lock()
                    // Higher precedence overrides lower
```

```
                    existing, exists := r.skills[skill.Name]
                    if !exists || skill.Level < existing.Level {
                        r.skills[skill.Name] = skill
                    }
                    r.mu.Unlock()
                }
            }
            return nil
        })
        if err != nil {
            return err
        }
    }
    return nil
}

// GetSkill loads full instructions on demand (progressive
        disclosure)
func (r *Registry) GetSkill(name string) (*Skill, error) {
    r.mu.RLock()
    skill, exists := r.skills[name]
    r.mu.RUnlock()

    if !exists {
        return nil, fmt.Errorf("skill not found: %s", name)
    }

    // Load full instructions on first access
    if skill.Instructions == "" {
        if err := r.loadFullSkill(skill); err != nil {
            return nil, err
        }
    }

    return skill, nil
}

// loadSkillMetadata loads only metadata (fast, for indexing)
func (r *Registry) loadSkillMetadata(dir string) (*Skill, error)
        {
    // Parse SKILL.md frontmatter
    // ... implementation
}

// loadFullSkill loads complete instructions + scripts +
        references
func (r *Registry) loadFullSkill(skill *Skill) error {
    // Read full SKILL.md content after frontmatter
```

```
    // Load scripts/ and references/ directories
    // ... implementation
    return nil
}

// Watch enables live change detection
func (r *Registry) Watch(ctx context.Context) error {
    // Use fsnotify to watch skill directories
    // Auto-reload on add/edit/remove
    // ... implementation
    return nil
}
```

New File: cmd/helix/plugin.go

```
package cmd

import (
    "fmt"
    "github.com/spf13/cobra"
)

var pluginCmd = &cobra.Command{
    Use:   "plugin",
    Short: "Manage HelixCode plugins and skills",
    Long:  "Install, update, and manage skills from curated
        marketplaces",
}

var pluginMarketplaceAddCmd = &cobra.Command{
    Use:   "marketplace add <repo>",
    Short: "Add a skill marketplace repository",
    RunE: func(cmd *cobra.Command, args []string) error {
        if len(args) < 1 {
            return fmt.Errorf("repository URL required")
        }
        repo := args[0]
        // Add marketplace to config
        return addMarketplace(repo)
    },
}

var pluginInstallCmd = &cobra.Command{
    Use:   "install <skill>[@<source>]",
    Short: "Install a skill or plugin",
    RunE: func(cmd *cobra.Command, args []string) error {
        if len(args) < 1 {
            return fmt.Errorf("skill name required")
        }
```

```
        // Install from marketplace or direct URL
        return installSkill(args[0])
    },
}

var pluginListCmd = &cobra.Command{
    Use:   "list",
    Short: "List installed skills",
    RunE: func(cmd *cobra.Command, args []string) error {
        registry := skills.NewRegistry()
        registry.Discover(cmd.Context())
        // Print table of skills
        return nil
    },
}

func init() {
    pluginCmd.AddCommand(pluginMarketplaceAddCmd)
    pluginCmd.AddCommand(pluginInstallCmd)
    pluginCmd.AddCommand(pluginListCmd)
    rootCmd.AddCommand(pluginCmd)
}
```

Modify: `internal/agent/swarm/coordinator.go`

```
// Add to existing Coordinator struct
type AgentTeam struct {
    ID          string
    LeadAgentID string
    Teammates   []string
    TaskList    *SharedTaskList
    Mailbox     *MailboxSystem
}

type SharedTaskList struct {
    Path    string // markdown file path
    Tasks   []TeamTask
    mu      sync.RWMutex
}

type TeamTask struct {
    ID          string
    Description string
    Status      string // pending, in-progress, complete, blocked
    Assignee    string
    Notes       string
}

type MailboxSystem struct {
```

```
    InboxPath string
    Messages  []TeamMessage
    mu        sync.RWMutex
}

type TeamMessage struct {
    ID        string
    From      string
    To        string
    Content   string
    Timestamp time.Time
}

// Add method to Coordinator
func (c *Coordinator) CreateAgentTeam(ctx context.Context,
        leadID string, teammateSpecs []AgentSpec) (*AgentTeam,
        error) {
    team := &AgentTeam{
        ID:          generateTeamID(),
        LeadAgentID: leadID,
        Teammates:   make([]string, 0, len(teammateSpecs)),
        TaskList:    &SharedTaskList{Path:
        generateTaskListPath()},
        Mailbox:     &MailboxSystem{InboxPath:
        generateMailboxPath()},
    }

    // Spawn teammates
    for _, spec := range teammateSpecs {
        agent, err := c.SpawnSubAgent(ctx, spec)
        if err != nil {
            return nil, err
        }
        team.Teammates = append(team.Teammates, agent.ID())
    }

    return team, nil
}
```

Porting Complexity: High

- Progressive disclosure requires careful context window management
- Plugin marketplace needs package management infrastructure
- Agent teams require file-based coordination primitives

Anti-Bluff Test

```
# Test 1: Skill discovery
helix plugin list
```

```
# Expected: Shows installed skills with metadata

# Test 2: Progressive disclosure
helix skill load "code-review"
# Expected: Only loads full instructions when invoked

# Test 3: Live reload
echo "---
name: test-skill
description: Test skill
---
# Instructions" > ~/.helix/skills/test/SKILL.md
helix skill list
# Expected: test-skill appears without restart

# Test 4: Agent team
cat > test_tasks.md << 'EOF'
- [ ] Task A: Implement auth
- [ ] Task B: Write tests
EOF
helix team create --tasks test_tasks.md --agents 3
# Expected: 3 agents claim tasks and complete independently
```

Integration Verification

- [ ] `internal/skills/registry.go` compiles
- [ ] `helix plugin --help` shows marketplace commands
- [ ] Skill hot-reload works across session
- [ ] Agent teams file coordination prevents race conditions
- [ ] Task list markdown format is human-readable

Agent 2: Bridle

> **Source**: `cli_agents/bridle` **Language**: TypeScript **License**: MIT

Feature Inventory - Top 3 Unique Features

1. Cross-Harness Configuration Manager - "Package manager" for AI coding harnesses (Claude Code, OpenCode, Goose, Copilot CLI, Crush, Droid) - Auto-translates paths, namings, schemas, configs between harnesses - Install skills/MCPs from any GitHub repo and auto-translate for target harness - Profile management: work/personal/minimal per harness

2. Harness Path Translation Matrix - Skills: `~/.claude/skills/` -> `~/.config/opencode/skill/` -> `~/.config/goose/skills/` - Agents: `~/.claude/plugins/*/agents/` -> `~/.config/opencode/agent/` - Commands: `~/.claude/plugins/*/commands/` -> `~/.config/opencode/command/` - MCPs: `~/.claude/.mcp.json` -> `opencode.jsonc` -> `config.yaml`

3. Profile System with Isolation - Create profiles from current config: `bridle profile create claude work --from-current` - Switch profiles: `bridle profile switch claude personal` - Diff profiles: `bridle profile diff claude work personal` - Profile = saved configuration snapshot per harness

HelixCode Integration

Feature	HelixCode Package	Integration Point
Cross-harness config	`internal/config/harness/`	New package - config translation layer
Profile system	`internal/config/profiles/`	Extend existing config with profiles
Path translation	`internal/config/translator.go`	Config path normalization

Exact Code Changes

New File: `internal/config/harness/translator.go`

```
package harness

import (
    "fmt"
    "path/filepath"
)

// HarnessType identifies the target agent harness
type HarnessType string

const (
    HarnessClaude     HarnessType = "claude"
    HarnessOpenCode   HarnessType = "opencode"
    HarnessGoose      HarnessType = "goose"
    HarnessCopilot    HarnessType = "copilot"
    HarnessCrush      HarnessType = "crush"
    HarnessDroid      HarnessType = "droid"
    HarnessHelix      HarnessType = "helix"
)

// ConfigPath maps component types to harness-specific paths
type ConfigPath struct {
    Component string      // skills, agents, commands, mcps
    Harness   HarnessType
```

```
    Path      string
    Format    string // json, yaml, toml, jsonc
}

// PathMatrix defines all known config paths per harness
var PathMatrix = map[HarnessType]map[string]ConfigPath{
    HarnessClaude: {
        "skills":   {Component: "skills", Harness:
        HarnessClaude, Path: "~/.claude/skills/", Format:
        "markdown"},
        "agents":   {Component: "agents", Harness:
        HarnessClaude, Path: "~/.claude/plugins/*/agents/",
        Format: "markdown"},
        "commands": {Component: "commands", Harness:
        HarnessClaude, Path: "~/.claude/plugins/*/commands/",
        Format: "markdown"},
        "mcps":     {Component: "mcps", Harness: HarnessClaude,
        Path: "~/.claude/.mcp.json", Format: "json"},
    },
    HarnessHelix: {
        "skills":   {Component: "skills", Harness: HarnessHelix,
        Path: "~/.helix/skills/", Format: "markdown"},
        "agents":   {Component: "agents", Harness: HarnessHelix,
        Path: "~/.helix/agents/", Format: "markdown"},
        "commands": {Component: "commands", Harness:
        HarnessHelix, Path: "~/.helix/commands/", Format:
        "markdown"},
        "mcps":     {Component: "mcps", Harness: HarnessHelix,
        Path: "~/.helix/mcp.json", Format: "json"},
    },
    // ... other harnesses
}

// Translator converts config between harness formats
type Translator struct {
    sourceHarness HarnessType
    targetHarness HarnessType
}

func NewTranslator(source, target HarnessType) *Translator {
    return &Translator{sourceHarness: source, targetHarness:
        target}
}

// TranslateSkills converts skills from source to target harness
        format
func (t *Translator) TranslateSkills(sourceDir string, targetDir
        string) error {
```

```
    sourcePaths := PathMatrix[t.sourceHarness]["skills"]
    targetPaths := PathMatrix[t.targetHarness]["skills"]

    // Walk source directory, translate each skill
    // Rename files, adjust frontmatter, rewrite paths
    // ... implementation

    return nil
}

// TranslateMCPs converts MCP config between JSON/YAML/TOML
        formats
func (t *Translator) TranslateMCPs(sourcePath string, targetPath
        string) error {
    // Parse source format, convert to target format
    // Handle schema differences between harnesses
    // ... implementation
    return nil
}
```

New File: cmd/helix/profile.go

```
package cmd

import "github.com/spf13/cobra"

var profileCmd = &cobra.Command{
    Use:   "profile",
    Short: "Manage HelixCode configuration profiles",
}

var profileCreateCmd = &cobra.Command{
    Use:   "create <name> [--from-current]",
    Short: "Create a new configuration profile",
    RunE: func(cmd *cobra.Command, args []string) error {
        name := args[0]
        fromCurrent, _ := cmd.Flags().GetBool("from-current")
        // Save current config as named profile
        return createProfile(name, fromCurrent)
    },
}

var profileSwitchCmd = &cobra.Command{
    Use:   "switch <name>",
    Short: "Activate a configuration profile",
    RunE: func(cmd *cobra.Command, args []string) error {
        return switchProfile(args[0])
    },
}
```

```
var profileDiffCmd = &cobra.Command{
    Use:   "diff <name1> [name2]",
    Short: "Compare configuration profiles",
    RunE: func(cmd *cobra.Command, args []string) error {
        // Show diff between profile snapshots
        return diffProfiles(args)
    },
}

func init() {
    profileCreateCmd.Flags().Bool("from-current", false, "Create
        from current config")
    profileCmd.AddCommand(profileCreateCmd)
    profileCmd.AddCommand(profileSwitchCmd)
    profileCmd.AddCommand(profileDiffCmd)
    rootCmd.AddCommand(profileCmd)
}
```

Porting Complexity: Medium

- Path translation is mechanical but requires comprehensive mapping
- Profile system is similar to existing config but needs snapshotting
- No algorithmic complexity - mostly data transformation

Anti-Bluff Test

```
# Test 1: Profile creation
helix profile create work --from-current
helix profile create personal --from-current
# Expected: Two profiles stored in ~/.helix/profiles/

# Test 2: Profile switching
helix profile switch personal
# Expected: Config reloads, different model/provider active

# Test 3: Profile diff
helix profile diff work personal
# Expected: Shows differences in model, MCPs, settings

# Test 4: Import from Claude Code
helix config import --from claude --skills --mcps
# Expected: Skills and MCPs translated to Helix paths
```

Agent 3: Codai

Source: `cli_agents/codai` **Language**: Python **License**: MIT

Feature Inventory - Top 3 Unique Features

1. Session-Based CLI Architecture - Session-based workflow: add features, refactor, code reviews - Deep project context understanding - analyzes entire codebase - Multi-LLM support: GPT-4o, GPT-4, Ollama, and more - VS Code and JetBrains plugin integration (planned)

2. Full Project Context Analysis - Scans and understands entire project structure - Context-aware suggestions based on codebase patterns - Multi-file coordination for large changes

3. IDE Plugin Bridge - Terminal CLI + IDE extension hybrid architecture - Shared context between terminal and IDE sessions - Planned VS Code and JetBrains support

HelixCode Integration

Feature	HelixCode Package	Integration Point
Session-based CLI	`internal/session/`	Extend with session templates
Project context analysis	`internal/context/builder.go`	Enhance with full-project scanning
IDE plugin bridge	`api/openapi.yaml`	Add IDE plugin endpoints

Porting Complexity: Low

- Session-based CLI already partially exists in HelixCode
- Project context is similar to existing context builder
- IDE plugin bridge is API-level work

Anti-Bluff Test

```
helix session start --template "code-review"
# Expected: Pre-loads review checklist and project context

helix context analyze --depth full
# Expected: Scans all files, builds dependency graph
```

Agent 4: Codename_Goose

Source: `cli_agents/codename-goose` **Language**: Rust **License**: Apache 2.0

Feature Inventory - Top 3 Unique Features

1. Deep MCP Integration (70+ Extensions) - One of earliest and deepest MCP adopters - 70+ documented MCP extensions - MCP Apps: interactive UIs rendered inside Goose Desktop (buttons, forms, visualizations) - Built-in desktop app + CLI + API

2. Recipes System - Capture workflows as portable YAML configs - Share with team, run in CI - Include instructions, extensions, parameters, subrecipes - Reusable automation templates

3. Subagents with Parallel Execution - Spawn independent subagents for parallel tasks - Code review, research, file processing - Main conversation stays clean - Security: prompt injection detection, tool permission controls, sandbox mode, adversary reviewer

HelixCode Integration

Feature	HelixCode Package	Integration Point
MCP Apps	`internal/mcp/apps/`	New - interactive UI in MCP responses
Recipes	`internal/workflow/recipes/`	Extend workflow with YAML recipes
Adversary reviewer	`internal/agent/security/`	New security agent role

Exact Code Changes

New File: `internal/workflow/recipes/recipe.go`

```
package recipes

import (
    "context"
    "fmt"
    "os"
    "gopkg.in/yaml.v3"
)

// Recipe is a portable workflow configuration
type Recipe struct {
    Name         string            `yaml:"name"`
    Description string            `yaml:"description"`
    Version     string            `yaml:"version"`
    Instructions []string          `yaml:"instructions"`
    Extensions  []ExtensionRef    `yaml:"extensions"`
    Parameters  []Parameter       `yaml:"parameters"`
    Subrecipes  []string          `yaml:"subrecipes"`
    Steps       []RecipeStep      `yaml:"steps"`
}

type ExtensionRef struct {
    Name    string `yaml:"name"`
    Version string `yaml:"version"`
}

type Parameter struct {
    Name        string `yaml:"name"`
    Type        string `yaml:"type"`
    Default     string `yaml:"default"`
```

```
    Description string `yaml:"description"`
}

type RecipeStep struct {
    Name        string            `yaml:"name"`
    Action      string            `yaml:"action"`
    Parameters  map[string]string `yaml:"parameters"`
    Condition   string            `yaml:"condition"`
}

// LoadRecipe loads a recipe from YAML file
func LoadRecipe(path string) (*Recipe, error) {
    data, err := os.ReadFile(path)
    if err != nil {
        return nil, err
    }

    var recipe Recipe
    if err := yaml.Unmarshal(data, &recipe); err != nil {
        return nil, err
    }

    return &recipe, nil
}

// Execute runs a recipe with given parameters
func (r *Recipe) Execute(ctx context.Context, params
        map[string]string) error {
    // Validate parameters
    // Execute each step in sequence
    // Handle subrecipes recursively
    // ... implementation
    return nil
}

// Save persists recipe to YAML file
func (r *Recipe) Save(path string) error {
    data, err := yaml.Marshal(r)
    if err != nil {
        return err
    }
    return os.WriteFile(path, data, 0644)
}
```

New File: `internal/agent/security/adversary.go`

```
package security

import (
```

```
    "context"
    "fmt"
    "strings"
)

// AdversaryReviewer watches agent actions for security issues
type AdversaryReviewer struct {
    rules []SecurityRule
}

type SecurityRule struct {
    Name        string
    Pattern     string
    Severity    string // critical, high, medium, low
    Description string
}

func NewAdversaryReviewer() *AdversaryReviewer {
    return &AdversaryReviewer{
        rules: []SecurityRule{
            {
                Name:        "command-injection",
                Pattern:     "`.*`.*\$\\(.*\\)|eval\\(|exec\\(",
                Severity:    "critical",
                Description: "Potential command injection in
        generated code",
            },
            {
                Name:        "data-exfiltration",
                Pattern:     "curl.*http|wget.*http|fetch\\
        (.*http",
                Severity:    "high",
                Description: "Potential data exfiltration
        attempt",
            },
            {
                Name:        "prompt-injection",
                Pattern:     "ignore.*previous|
        disregard.*instructions|forget.*rules",
                Severity:    "high",
                Description: "Potential prompt injection
        attempt",
            },
        },
    }
}

// ReviewAction analyzes an agent action for security issues
```

```
func (a *AdversaryReviewer) ReviewAction(ctx context.Context,
        action string, tool string) (*SecurityReview, error) {
    review := &SecurityReview{
        Action:  action,
        Tool:    tool,
        Issues:  []SecurityIssue{},
        Status:  "pass",
    }

    for _, rule := range a.rules {
        if strings.Contains(action, rule.Pattern) ||
        matchesPattern(action, rule.Pattern) {
            issue := SecurityIssue{
                Rule:     rule.Name,
                Severity: rule.Severity,
                Description: rule.Description,
            }
            review.Issues = append(review.Issues, issue)
            if rule.Severity == "critical" {
                review.Status = "fail"
            } else if review.Status == "pass" {
                review.Status = "warn"
            }
        }
    }

    return review, nil
}

type SecurityReview struct {
    Action string
    Tool   string
    Status string // pass, warn, fail
    Issues []SecurityIssue
}

type SecurityIssue struct {
    Rule        string
    Severity    string
    Description string
}

func matchesPattern(text, pattern string) bool {
    // Regex matching implementation
    return false
}
```

Porting Complexity: High

- MCP Apps require UI rendering infrastructure
- Recipes need workflow engine integration
- Adversary reviewer needs comprehensive security rule sets

Anti-Bluff Test

```
# Test 1: Recipe execution
helix recipe run deploy-production.yml --param env=prod
# Expected: Executes workflow steps with extensions

# Test 2: MCP App rendering
# MCP server returns interactive form
# Helix renders it in terminal TUI

# Test 3: Adversary reviewer
helix security review --action "eval(user_input)"
# Expected: FAIL with command injection warning
```

Agent 5: Conduit

Source: `cli_agents/conduit` **Language**: TypeScript **License**: Unknown

Feature Inventory - Top 3 Unique Features

1. Visual Workflow Builder - Drag-and-drop workflow canvas - Block-based data processing - Join by Key, Transpose, Add Column blocks

2. Data Integration Workflows - Pull data from Facebook Ads, Google Ads - Weekly Ad Spend report generation - Column transformation and metric addition

3. Workflow Automation - Trigger-based execution - Delay/wait actions - AI agent injection into conversations

HelixCode Integration

Feature	HelixCode Package	Integration Point
Visual workflows	`applications/desktop/`	Extend Fyne app with workflow canvas
Data blocks	`internal/workflow/blocks/`	New block-based data processing
Triggers	`internal/workflow/triggers/`	Event-based workflow execution

Porting Complexity: Medium

- Visual workflow builder is UI-heavy

- Data blocks are straightforward transformations
- Trigger system can leverage existing event bus

Anti-Bluff Test

```
# Not CLI-testable; requires desktop app verification
```

Agent 6: DeepSeek_CLI

> **Source**: `cli_agents/deepseek-cli` **Language**: Python **License**: MIT

Feature Inventory - Top 3 Unique Features

1. DeepSeek Model Specialization - Optimized for DeepSeek-V3, DeepSeek-Coder - Simple, lightweight interface - Chat and code generation modes

2. Alternative Implementation (deepseek-cli-youkpan) - Fork with different UX patterns - Alternative config management

3. Minimal Footprint - Very small codebase - Easy to understand and modify

HelixCode Integration

Feature	HelixCode Package	Integration Point
DeepSeek support	`internal/llm/`	Already supported via factory
Lightweight CLI	`cmd/helix/`	Reference for minimal CLI patterns

Porting Complexity: Low

- HelixCode already supports DeepSeek via LLM factory
- Minimal features - mostly model access patterns
- Priority: LOW - feature overlap with existing provider support

Anti-Bluff Test

```
helix llm providers | grep -i deepseek
# Expected: DeepSeek listed as available provider
```

Agent 7: Emdash

> **Source**: `cli_agents/emdash` (or external: generalaction/emdash) **Language**: TypeScript/Electron **License**: Apache 2.0 (with patent grant)

Feature Inventory - Top 3 Unique Features

1. Agentic Development Environment (ADE) - Desktop app for running 24+ CLI agents in parallel - Each agent isolated in its own git worktree - Provider-agnostic: Claude Code, Codex, Gemini, Amp, etc.

2. Parallel Agent Orchestration - Kanban view of running agents - Diff review across agents side-by-side - Issue integration: Linear, Jira, GitHub, GitLab, Forgejo - CI/CD checks monitoring inside the app

3. Remote Development via SSH - Connect to remote machines over SSH/SFTP - Run agents on remote projects with same worktree isolation - SSH agent and key authentication - Tmux sessions for long-running agents

HelixCode Integration

Feature	HelixCode Package	Integration Point
ADE concept	`applications/desktop/`	Extend Fyne app with agent orchestration
Worktree isolation	`internal/git/worktree.go`	New - git worktree management
Issue integration	`internal/integrations/issues/`	New - Linear/Jira/GitHub connectors
Remote SSH	`internal/worker/`	Extend existing SSH worker pool
Kanban view	`applications/desktop/`	New UI component

Exact Code Changes

New File: `internal/git/worktree.go`

```
package git

import (
    "context"
    "fmt"
    "os"
    "os/exec"
    "path/filepath"
    "strings"
    "sync"
)

// WorktreeManager manages isolated git worktrees for parallel
        agents
type WorktreeManager struct {
    mu        sync.Mutex
    baseRepo  string
    worktrees map[string]*Worktree
}

type Worktree struct {
    ID       string
    Name     string
    Path     string
```

```
    Branch   string
    AgentID  string
    Status   WorktreeStatus
}

type WorktreeStatus string

const (
    WorktreeStatusActive    WorktreeStatus = "active"
    WorktreeStatusIdle      WorktreeStatus = "idle"
    WorktreeStatusCleaning  WorktreeStatus = "cleaning"
    WorktreeStatusRemoved   WorktreeStatus = "removed"
)

// NewWorktreeManager creates a worktree manager for a base
        repository
func NewWorktreeManager(baseRepo string) (*WorktreeManager,
        error) {
    absPath, err := filepath.Abs(baseRepo)
    if err != nil {
        return nil, err
    }
    return &WorktreeManager{
        baseRepo:  absPath,
        worktrees: make(map[string]*Worktree),
    }, nil
}

// CreateWorktree creates a new isolated worktree for an agent
func (w *WorktreeManager) CreateWorktree(ctx context.Context,
        name string, agentID string) (*Worktree, error) {
    w.mu.Lock()
    defer w.mu.Unlock()

    id := generateWorktreeID(name)
    branchName := fmt.Sprintf("agent-%s", id)
    path := filepath.Join(w.baseRepo, ".worktrees", id)

    // Create branch
    cmd := exec.CommandContext(ctx, "git", "branch", branchName)
    cmd.Dir = w.baseRepo
    if err := cmd.Run(); err != nil {
        return nil, fmt.Errorf("failed to create branch: %w",
        err)
    }

    // Create worktree
```

```
    cmd = exec.CommandContext(ctx, "git", "worktree", "add",
        path, branchName)
    cmd.Dir = w.baseRepo
    if err := cmd.Run(); err != nil {
        return nil, fmt.Errorf("failed to create worktree: %w",
        err)
    }

    wt := &Worktree{
        ID:       id,
        Name:     name,
        Path:     path,
        Branch:   branchName,
        AgentID: agentID,
        Status:   WorktreeStatusActive,
    }
    w.worktrees[id] = wt

    return wt, nil
}

// RemoveWorktree removes a worktree and its branch
func (w *WorktreeManager) RemoveWorktree(ctx context.Context, id
        string) error {
    w.mu.Lock()
    wt, exists := w.worktrees[id]
    w.mu.Unlock()

    if !exists {
        return fmt.Errorf("worktree not found: %s", id)
    }

    wt.Status = WorktreeStatusCleaning

    // Remove worktree
    cmd := exec.CommandContext(ctx, "git", "worktree", "remove",
        wt.Path)
    cmd.Dir = w.baseRepo
    if err := cmd.Run(); err != nil {
        // Force remove if needed
        os.RemoveAll(wt.Path)
    }

    // Delete branch
    cmd = exec.CommandContext(ctx, "git", "branch", "-D",
        wt.Branch)
    cmd.Dir = w.baseRepo
    cmd.Run()
```

```
    w.mu.Lock()
    delete(w.worktrees, id)
    w.mu.Unlock()

    return nil
}

// ListWorktrees returns all managed worktrees
func (w *WorktreeManager) ListWorktrees() []*Worktree {
    w.mu.Lock()
    defer w.mu.Unlock()

    result := make([]*Worktree, 0, len(w.worktrees))
    for _, wt := range w.worktrees {
        result = append(result, wt)
    }
    return result
}

func generateWorktreeID(name string) string {
    // Sanitize name + add timestamp
    sanitized := strings.ReplaceAll(name, " ", "-")
    sanitized = strings.ReplaceAll(sanitized, "/", "-")
    return fmt.Sprintf("%s-%d", sanitized, time.Now().Unix())
}
```

New File: `internal/integrations/issues/connector.go`

```
package issues

import "context"

// IssueConnector provides unified access to issue trackers
type IssueConnector interface {
    // ListIssues returns issues matching criteria
    ListIssues(ctx context.Context, opts ListOptions) ([]Issue,
        error)

    // GetIssue returns a single issue by ID
    GetIssue(ctx context.Context, id string) (*Issue, error)

    // GetIssueContext returns issue content formatted for agent
        consumption
    GetIssueContext(ctx context.Context, id string) (string,
        error)

    // Provider returns the provider name
    Provider() string
```

```
}

type Issue struct {
    ID          string
    Title       string
    Description string
    Status      string
    Labels      []string
    Assignee    string
    URL         string
    Metadata    map[string]interface{}
}

type ListOptions struct {
    Status   string
    Labels   []string
    Assignee string
    Limit    int
}

// ConnectorRegistry manages all issue connectors
type ConnectorRegistry struct {
    connectors map[string]IssueConnector
}

func NewConnectorRegistry() *ConnectorRegistry {
    return &ConnectorRegistry{
        connectors: make(map[string]IssueConnector),
    }
}

func (r *ConnectorRegistry) Register(name string, connector
        IssueConnector) {
    r.connectors[name] = connector
}

func (r *ConnectorRegistry) Get(name string) (IssueConnector,
        bool) {
    c, ok := r.connectors[name]
    return c, ok
}
```

Porting Complexity: High

- Desktop app requires Electron-equivalent (Fyne) implementation
- Worktree management needs careful git state handling
- Issue integration requires multiple API connectors

Anti-Bluff Test

```
# Test 1: Worktree creation
helix worktree create --agent "test-agent-1"
helix worktree create --agent "test-agent-2"
git worktree list
# Expected: Two worktrees listed

# Test 2: Parallel agents
helix agent spawn --worktree wt-1 --task "Implement auth"
helix agent spawn --worktree wt-2 --task "Write tests"
# Expected: Both agents run in isolated worktrees

# Test 3: Issue import
helix issue import --provider github --repo "my-org/my-repo" --
        limit 5
# Expected: Issues loaded into agent context

# Test 4: Worktree cleanup
helix worktree remove wt-1
git worktree list
# Expected: Worktree removed, branch deleted
```

Agent 8: FauxPilot

Source: `cli_agents/fauxpilot` **Language**: Python **License**: MIT

Feature Inventory - Top 3 Unique Features

1. Self-Hosted Code Completion - Alternative to GitHub Copilot that runs locally - Salesforce CodeGen models on NVIDIA Triton Inference Server - FasterTransformer backend for local inference

2. Air-Gapped Deployment - No external data transmission - Perfect for sensitive codebases - Requires Docker + NVIDIA GPU

3. Copilot-Style API Compatibility - Compatible with Copilot-style APIs - Drop-in replacement for teams with privacy needs

HelixCode Integration

Feature	HelixCode Package	Integration Point
Self-hosted completion	`internal/llm/local/`	Extend local provider support
GPU inference	`internal/hardware/`	Leverage GPU detection
Completion API	`api/openapi.yaml`	Add completion endpoints

Porting Complexity: Medium

- Local inference already partially supported (Ollama, etc.)
- Need to add CodeGen model support and Triton integration
- Completion API is new endpoint type

Anti-Bluff Test

```
helix llm local status
# Expected: Shows local model status, GPU info

helix complete --model codegen-16b --file main.go
# Expected: Returns completion suggestions
```

Agent 9: Gemini_CLI

Source: `cli_agents/gemini-cli` **Language**: TypeScript **License**: Apache 2.0

Feature Inventory - Top 3 Unique Features

1. Native Gemini Model Integration - Google's official CLI for Gemini models - Direct API access to latest Gemini updates - Optimized for Gemini's 1M+ context window

2. Streaming Response Architecture - Real-time token streaming - Progress indication for long operations - SSE-based streaming endpoints

3. Multi-Modal Input Support - Image input for code analysis - PDF/document processing - Audio input (future)

HelixCode Integration

Feature	HelixCode Package	Integration Point
Gemini provider	`internal/llm/`	Already supported - verify full integration
Streaming	`internal/llm/streaming/`	Enhance streaming infrastructure
Multi-modal	`internal/llm/vision/`	Extend vision support

Porting Complexity: Low

- Gemini already supported in LLM factory
- Streaming already exists
- Vision support partially implemented
- Priority: LOW - mostly verification work

Anti-Bluff Test

```
helix llm providers | grep -i gemini
# Expected: Gemini listed with models
```

```
helix chat --provider gemini --model gemini-2.5-pro
        "Explain this code" @screenshot.png
# Expected: Multi-modal response
```

Agent 10: Get-Shit-Done

> **Source**: `cli_agents/get-shit-done` **Language**: Unknown **License**: Unknown

Feature Inventory - Top 3 Unique Features

1. Productivity-Focused CLI - Task management and focus enhancement - Distraction blocking - Pomodoro-style work sessions

2. Goal-Oriented Interface - Set daily/weekly goals - Track progress - Completion celebration

3. Minimal Distraction Design - Clean, focused UI - No unnecessary features

HelixCode Integration

Feature	HelixCode Package	Integration Point
Task tracking	`internal/task/`	Already exists - extend with pomodoro
Focus mode	`internal/session/`	Add focus session mode

Porting Complexity: Low

- Minimal feature set
- Can be implemented as session mode extension

Anti-Bluff Test

```
helix session focus --duration 25m --goal "Implement feature X"
# Expected: Session enters focus mode with timer
```

Agent 11: GitHub-Copilot-CLI

> **Source**: `cli_agents/copilot-cli` **Language**: TypeScript **License**: Proprietary (GitHub)

Feature Inventory - Top 3 Unique Features

1. Plan Mode with Shift+Tab Toggle - Interactive plan mode: AI suggests but doesn't execute - Toggle between ask/execute and plan modes with Shift+Tab - Structured implementation planning before code writing

2. Context Management Commands - `/compact` - manual context compression - `/context` - token usage breakdown - Auto-compaction at 95% token limit - Virtually infinite sessions via compression

3. Custom Agent System - `.github/copilot-instructions.md` - repo-wide instructions - `.github/instructions/**/*.instructions.md` - path-specific - `AGENTS.md` - custom agent definitions - `/agent` slash command to invoke custom agents - `--agent=<name>` CLI option

HelixCode Integration

Feature	HelixCode Package	Integration Point
Plan mode toggle	`internal/session/modes.go`	Add plan mode with Shift+Tab
Context compaction	`internal/llm/compression/`	Enhance auto-compaction
Custom agents	`internal/agent/custom/`	New custom agent loader
Instructions files	`internal/context/providers/`	Add file-based instructions

Exact Code Changes

Modify: `internal/session/modes.go`

```
package session

// SessionMode defines the operating mode
type SessionMode string

const (
    ModeAsk       SessionMode = "ask"
    ModePlan      SessionMode = "plan"
    ModeExecute   SessionMode = "execute"
    ModeArchitect SessionMode = "architect"
)

type ModeController struct {
    currentMode SessionMode
    modes       map[SessionMode]*ModeConfig
}

type ModeConfig struct {
    Mode        SessionMode
    Name        string
    Description string
    AutoApprove []string // tools auto-approved in this mode
    ReadOnly    bool       // plan mode is read-only
}
```

```
func NewModeController() *ModeController {
    mc := &ModeController{
        currentMode: ModeAsk,
        modes: map[SessionMode]*ModeConfig{
            ModeAsk: {
                Mode:        ModeAsk,
                Name:        "Ask/Execute",
                Description: "Interactive coding with tool
        approval",
                AutoApprove: []string{},
                ReadOnly:    false,
            },
            ModePlan: {
                Mode:        ModePlan,
                Name:        "Plan",
                Description: "AI plans but does not execute",
                AutoApprove: []string{"read_file", "grep",
        "glob"},
                ReadOnly:    true,
            },
            ModeExecute: {
                Mode:        ModeExecute,
                Name:        "Execute",
                Description: "Auto-approve safe tools",
                AutoApprove: []string{"read_file", "write_file",
        "bash"},
                ReadOnly:    false,
            },
        },
    }
    return mc
}

// ToggleMode switches between modes (bound to Shift+Tab)
func (mc *ModeController) ToggleMode() SessionMode {
    modes := []SessionMode{ModeAsk, ModePlan, ModeExecute}
    for i, m := range modes {
        if m == mc.currentMode {
            mc.currentMode = modes[(i+1)%len(modes)]
            return mc.currentMode
        }
    }
    return mc.currentMode
}

// IsReadOnly returns true if current mode doesn't execute
func (mc *ModeController) IsReadOnly() bool {
```

```
    config, exists := mc.modes[mc.currentMode]
    return exists && config.ReadOnly
}
```

New File: `internal/agent/custom/loader.go`

```
package custom

import (
    "context"
    "fmt"
    "os"
    "path/filepath"
    "strings"
    "gopkg.in/yaml.v3"
)

// CustomAgent represents a user-defined agent
type CustomAgent struct {
    Name        string            `yaml:"name"`
    Description string            `yaml:"description"`
    SystemPrompt string           `yaml:"system_prompt"`
    Tools       []string          `yaml:"tools"`
    Model       string            `yaml:"model"`
    Instructions []string         `yaml:"instructions"`
}

// Loader discovers and loads custom agents from project files
type Loader struct {
    searchPaths []string
}

func NewLoader() *Loader {
    return &Loader{
        searchPaths: []string{
            ".github/copilot-instructions.md",
            ".github/instructions/",
            "AGENTS.md",
            ".helix/agents/",
        },
    }
}

// LoadAgents discovers all custom agents in the project
func (l *Loader) LoadAgents(ctx context.Context, projectPath
        string) ([]*CustomAgent, error) {
    var agents []*CustomAgent

    for _, searchPath := range l.searchPaths {
```

```
        fullPath := filepath.Join(projectPath, searchPath)

        if strings.HasSuffix(searchPath, ".md") {
            // Single file
            if agent := l.loadFromMarkdown(fullPath); agent !=
        nil {
                agents = append(agents, agent)
            }
        } else {
            // Directory
            entries, err := os.ReadDir(fullPath)
            if err != nil {
                continue
            }
            for _, entry := range entries {
                if strings.HasSuffix(entry.Name(), ".md") {
                    path := filepath.Join(fullPath, entry.Name())
                    if agent := l.loadFromMarkdown(path); agent !
        = nil {
                        agents = append(agents, agent)
                    }
                }
            }
        }
    }

    return agents, nil
}

func (l *Loader) loadFromMarkdown(path string) *CustomAgent {
    data, err := os.ReadFile(path)
    if err != nil {
        return nil
    }

    // Parse frontmatter if present
    content := string(data)
    if strings.HasPrefix(content, "---") {
        // Extract YAML frontmatter
        endIdx := strings.Index(content[3:], "---")
        if endIdx > 0 {
            frontmatter := content[3 : endIdx+3]
            var agent CustomAgent
            if err := yaml.Unmarshal([]byte(frontmatter),
        &agent); err == nil && agent.Name != "" {
                agent.Instructions = []string{content[endIdx+6:]}
                return &agent
            }
```

```
        }
    }

    return nil
}
```

Porting Complexity: Medium

- Plan mode is conceptually simple but needs UI integration
- Context compaction exists but needs auto-trigger at 95%
- Custom agents require file watching and hot-reload

Anti-Bluff Test

```
# Test 1: Plan mode toggle
helix session start
# Press Shift+Tab
# Expected: Mode switches Ask -> Plan -> Execute -> Ask

# Test 2: Plan mode behavior
# In plan mode, ask "Create a REST API"
# Expected: AI outputs plan but does NOT create files

# Test 3: Custom agent
mkdir -p .helix/agents/
cat > .helix/agents/security-reviewer.md << 'EOF'
---
name: security-reviewer
description: Reviews code for security issues
tools: [read_file, grep]
model: claude-sonnet-4
---
You are a security expert. Review code for vulnerabilities.
EOF
helix agent list
# Expected: security-reviewer appears in agent list

# Test 4: Context compaction
helix context usage
# Expected: Shows token breakdown per file
```

Agent 12: GitHub-Spec-Kit

Source: `cli_agents/spec-kit` **Language**: Ruby **License**: MIT

Feature Inventory - Top 3 Unique Features

1. Specification Management - Structured spec documents for projects - GitHub integration for spec tracking - Structured data handling for requirements

2. GitHub Integration - Spec-to-issue linking - PR templates from specs - Review checklists

3. Data Structure Templates - Standardized spec formats - YAML/JSON schema validation

HelixCode Integration

Feature	HelixCode Package	Integration Point
Spec management	`internal/specs/`	New package for spec documents
GitHub linking	`internal/integrations/github/`	Extend GitHub integration

Porting Complexity: Low

- Primarily document management
- Can leverage existing GitHub integration

Anti-Bluff Test

```
helix spec create --template api "User Authentication API"
# Expected: Creates structured spec document
```

Agent 13: GitMCP

Source: `cli_agents/git-mcp` **Language**: TypeScript **License**: MIT

Feature Inventory - Top 3 Unique Features

1. Git Operations via MCP - Full Git operations through Model Context Protocol - init, clone, status, add, commit, push, pull - Branch management: list, create, delete, checkout

2. GitHub Integration via MCP - Built-in GitHub support via Personal Access Token - PR management through MCP tools - Issue tracking

3. Bulk Git Actions - Execute multiple Git operations in sequence - Repository caching for performance - Smart path handling

HelixCode Integration

Feature	HelixCode Package	Integration Point
Git MCP tools	`internal/mcp/tools/git.go`	New MCP tool definitions
GitHub MCP		New GitHub MCP tools

Feature	HelixCode Package	Integration Point
	`internal/mcp/tools/github.go`	
Bulk actions	`internal/tools/git/`	Extend git tool with batching

Porting Complexity: Medium

- MCP tools are straightforward to define
- Git operations already exist in HelixCode
- Need to expose through MCP protocol

Anti-Bluff Test

```
# Test via MCP client
# mcp tool: git_status -> returns git status as MCP response
# mcp tool: git_commit -> commits with generated message
```

Agent 14: Mistral_Code

> **Source**: `cli_agents/mistral-code` **Language**: TypeScript **License**: Apache 2.0

Feature Inventory - Top 3 Unique Features

1. Devstral 2 Model Integration - State-of-the-art open-source coding model - 256K context window, up to 1M with extrapolation - Apache 2.0 license

2. Vibe CLI Terminal Agent - Interactive chat with @file autocomplete - Stateful terminal (bash) - Todo list management - Ask user questions with options

3. Agent Modes with Permissions - `default`: requires approval - `plan`: read-only, auto-approves safe tools - `accept-edits`: auto-approves file edits - `auto-approve`: all tools (caution) - Toggle with Shift+Tab

HelixCode Integration

Feature	HelixCode Package	Integration Point
Mistral provider	`internal/llm/`	Already exists - verify
Vibe CLI UX	`applications/terminal_ui/`	Reference for UX patterns
Agent modes	`internal/session/modes.go`	Already covered in Copilot CLI
Todo list	`internal/tools/todo.go`	New - task tracking within session

Exact Code Changes

New File: `internal/tools/todo.go`

```
package tools

import (
    "context"
    "fmt"
    "sync"
    "time"
)

// TodoManager tracks agent tasks within a session
type TodoManager struct {
    mu    sync.RWMutex
    items []TodoItem
}

type TodoItem struct {
    ID          string
    Description string
    Status      TodoStatus
    CreatedAt   time.Time
    CompletedAt *time.Time
}

type TodoStatus string

const (
    TodoPending    TodoStatus = "pending"
    TodoInProgress TodoStatus = "in_progress"
    TodoComplete   TodoStatus = "complete"
    TodoBlocked    TodoStatus = "blocked"
)

func NewTodoManager() *TodoManager {
    return &TodoManager{
        items: make([]TodoItem, 0),
    }
}

// Add creates a new todo item
func (t *TodoManager) Add(description string) string {
    t.mu.Lock()
    defer t.mu.Unlock()

    id := fmt.Sprintf("todo-%d", len(t.items)+1)
    item := TodoItem{
```

```
        ID:          id,
        Description: description,
        Status:      TodoPending,
        CreatedAt:   time.Now(),
    }
    t.items = append(t.items, item)
    return id
}

// UpdateStatus changes item status
func (t *TodoManager) UpdateStatus(id string, status TodoStatus)
        error {
    t.mu.Lock()
    defer t.mu.Unlock()

    for i := range t.items {
        if t.items[i].ID == id {
            t.items[i].Status = status
            if status == TodoComplete {
                now := time.Now()
                t.items[i].CompletedAt = &now
            }
            return nil
        }
    }
    return fmt.Errorf("todo item not found: %s", id)
}

// List returns all items, optionally filtered by status
func (t *TodoManager) List(statusFilter ...TodoStatus)
        []TodoItem {
    t.mu.RLock()
    defer t.mu.RUnlock()

    if len(statusFilter) == 0 {
        result := make([]TodoItem, len(t.items))
        copy(result, t.items)
        return result
    }

    filterMap := make(map[TodoStatus]bool)
    for _, s := range statusFilter {
        filterMap[s] = true
    }

    var result []TodoItem
    for _, item := range t.items {
        if filterMap[item.Status] {
```

```
            result = append(result, item)
        }
    }
    return result
}

// ClearCompleted removes all completed items
func (t *TodoManager) ClearCompleted() {
    t.mu.Lock()
    defer t.mu.Unlock()

    var active []TodoItem
    for _, item := range t.items {
        if item.Status != TodoComplete {
            active = append(active, item)
        }
    }
    t.items = active
}

// Format returns a formatted todo list for display
func (t *TodoManager) Format() string {
    t.mu.RLock()
    defer t.mu.RUnlock()

    if len(t.items) == 0 {
        return "No todo items."
    }

    var output string
    output += fmt.Sprintf("Todo List (%d items):\n",
        len(t.items))
    for _, item := range t.items {
        symbol := "[ ]"
        if item.Status == TodoComplete {
            symbol = "[x]"
        } else if item.Status == TodoInProgress {
            symbol = "[~]"
        } else if item.Status == TodoBlocked {
            symbol = "[!]"
        }
        output += fmt.Sprintf("  %s %s: %s\n", symbol, item.ID,
        item.Description)
    }
    return output
}
```

Porting Complexity: Low-Medium

- Mistral provider already exists
- Todo manager is simple
- Agent modes already covered

Anti-Bluff Test

```
helix todo add "Implement authentication"
helix todo add "Write unit tests"
helix todo list
# Expected: Shows todo list with IDs

helix todo update todo-1 in_progress
helix todo update todo-2 complete
helix todo clear-completed
# Expected: Completed items removed
```

Agent 15: MobileAgent

Source: `cli_agents/mobile-agent` **Language**: Python **License**: Unknown

Feature Inventory - Top 3 Unique Features

1. Mobile UI Automation - Visual features with OCR and icon detection - XML-based UI component enumeration - Screenshot-based decision making

2. Mobile App Testing - AppAgent v2: flexible mobile interactions - Automated Android app interactions - GPT-4V-based visual understanding

3. Mobile-First Context - Device context (screen size, orientation) - App state awareness - Touch gesture simulation

HelixCode Integration

Feature	HelixCode Package	Integration Point
Mobile automation	`internal/tools/mobile/`	New - mobile device control
Screenshot analysis	`internal/llm/vision/`	Extend vision for UI analysis
Device connection	`internal/tools/mobile/adb.go`	ADB integration

Porting Complexity: High

- Mobile automation requires ADB/USB infrastructure
- Vision models for UI understanding
- Touch simulation needs platform-specific code

Anti-Bluff Test

```
# Requires physical device or emulator
helix mobile connect --device emulator-5554
helix mobile screenshot --analyze "Click the login button"
# Expected: Returns tap coordinates for login button
```

Agent 16: Multiagent-Coding-System

> **Source**: `cli_agents/multiagent-coding` **Language**: TypeScript **License**: Unknown

Feature Inventory - Top 3 Unique Features

1. Multi-Agent Orchestration Patterns - Orchestrator-worker pattern - Pipeline/sequential pattern - Hierarchical management - Swarm/collaborative pattern

2. Agent Communication Protocols - Shared state coordination - Message passing between agents - Handoff-based routing - A2A (Agent-to-Agent) protocol

3. Cross-Agent Verification - Verifier agent at handoff points - Schema checks on inter-agent messages - Consistency validation - Cost tracking per agent

HelixCode Integration

Feature	HelixCode Package	Integration Point
Orchestration patterns	`internal/agent/swarm/`	Extend with pattern registry
Agent communication	`internal/agent/comm/`	New - inter-agent messaging
Verification	`internal/agent/verifier/`	New - cross-agent validation

Porting Complexity: High

- Requires significant architecture for agent coordination
- Communication protocols need standardization
- Verification adds overhead but prevents error propagation

Anti-Bluff Test

```
helix swarm create --pattern orchestrator-worker --agents 3
# Expected: Creates orchestrator + 2 workers

helix swarm message --from agent-1 --to agent-2 "Task complete"
# Expected: Message delivered to agent-2
```

Agent 17: Nanocoder

> **Source**: `cli_agents/nanocoder` **Language**: TypeScript **License**: MIT (community collective)

Feature Inventory - Top 3 Unique Features

1. Development Modes (4-tier) - Normal: confirm each tool - Auto-Accept: most tools execute immediately - Yolo: every tool executes immediately (no exceptions) - Plan: tools shown but never executed - Toggle with Shift+Tab

2. Checkpointing System - Save conversation snapshots - Restore to any checkpoint - Branch from checkpoints - Named checkpoints

3. Custom Commands as Markdown - `.nanocoder/commands/` with YAML frontmatter - Template variables with `{{parameter}}` syntax - Namespaced by directory - Built-in: `/test`, `/review`, `/refactor:dry`, `/refactor:solid`

HelixCode Integration

Feature	HelixCode Package	Integration Point
4-tier modes	`internal/session/modes.go`	Extend with Yolo mode
Checkpointing	`internal/memory/checkpoint.go`	New - conversation snapshots
Custom commands	`internal/commands/custom.go`	Extend with markdown commands

Exact Code Changes

New File: `internal/memory/checkpoint.go`

```
package memory

import (
    "context"
    "fmt"
    "time"
)

// CheckpointManager handles conversation snapshots
type CheckpointManager struct {
    checkpoints map[string]*Checkpoint
}

type Checkpoint struct {
    ID          string
    Name        string
    SessionID   string
```

```
    CreatedAt   time.Time
    Messages    []MessageSnapshot
    Context     map[string]interface{}
    Description string
}

type MessageSnapshot struct {
    Role      string
    Content   string
    Timestamp time.Time
}

func NewCheckpointManager() *CheckpointManager {
    return &CheckpointManager{
        checkpoints: make(map[string]*Checkpoint),
    }
}

// Save creates a checkpoint from current conversation state
func (cm *CheckpointManager) Save(ctx context.Context, sessionID
        string, name string, description string, currentMessages
        []ChatMessage) (*Checkpoint, error) {
    id := fmt.Sprintf("cp-%d", time.Now().Unix())

    snapshots := make([]MessageSnapshot, len(currentMessages))
    for i, msg := range currentMessages {
        snapshots[i] = MessageSnapshot{
            Role:      msg.Role,
            Content:   msg.Content,
            Timestamp: msg.Timestamp,
        }
    }

    cp := &Checkpoint{
        ID:          id,
        Name:        name,
        SessionID:   sessionID,
        CreatedAt:   time.Now(),
        Messages:    snapshots,
        Description: description,
    }

    cm.checkpoints[id] = cp
    return cp, nil
}

// Restore returns messages from a checkpoint
```

```
func (cm *CheckpointManager) Restore(checkpointID string)
        ([]MessageSnapshot, error) {
    cp, exists := cm.checkpoints[checkpointID]
    if !exists {
        return nil, fmt.Errorf("checkpoint not found: %s",
        checkpointID)
    }
    return cp.Messages, nil
}

// List returns all checkpoints for a session
func (cm *CheckpointManager) List(sessionID string)
        []*Checkpoint {
    var result []*Checkpoint
    for _, cp := range cm.checkpoints {
        if cp.SessionID == sessionID {
            result = append(result, cp)
        }
    }
    return result
}

// Delete removes a checkpoint
func (cm *CheckpointManager) Delete(checkpointID string) error {
    if _, exists := cm.checkpoints[checkpointID]; !exists {
        return fmt.Errorf("checkpoint not found: %s",
        checkpointID)
    }
    delete(cm.checkpoints, checkpointID)
    return nil
}
```

New File: internal/commands/custom.go

```
package commands

import (
    "context"
    "fmt"
    "os"
    "path/filepath"
    "strings"
    "gopkg.in/yaml.v3"
)

// CustomCommand is a user-defined command loaded from markdown
type CustomCommand struct {
    Name        string            `yaml:"name"`
    Description string            `yaml:"description"`
```

```
    Aliases     []string          `yaml:"aliases"`
    Parameters  []CommandParameter `yaml:"parameters"`
    Prompt      string            // Content after frontmatter
    Namespace   string            // Directory name
}

type CommandParameter struct {
    Name        string `yaml:"name"`
    Type        string `yaml:"type"`
    Default     string `yaml:"default"`
    Description string `yaml:"description"`
    Required    bool   `yaml:"required"`
}

// CustomCommandLoader discovers and loads custom commands
type CustomCommandLoader struct {
    searchPaths []string
}

func NewCustomCommandLoader() *CustomCommandLoader {
    return &CustomCommandLoader{
        searchPaths: []string{
            ".helix/commands/",
            filepath.Join(os.Getenv("HOME"), ".helix/commands/"),
        },
    }
}

// LoadCommands discovers all custom commands
func (c *CustomCommandLoader) LoadCommands(ctx context.Context)
        ([]*CustomCommand, error) {
    var commands []*CustomCommand

    for _, searchPath := range c.searchPaths {
        entries, err := os.ReadDir(searchPath)
        if err != nil {
            continue
        }

        for _, entry := range entries {
            if entry.IsDir() {
                // Namespace directory
                nsEntries, _ :=
        os.ReadDir(filepath.Join(searchPath, entry.Name()))
                for _, nsEntry := range nsEntries {
                    if strings.HasSuffix(nsEntry.Name(), ".md") {
                        path := filepath.Join(searchPath,
        entry.Name(), nsEntry.Name())
```

```
                        if cmd := c.loadFromFile(path,
        entry.Name()); cmd != nil {
                            commands = append(commands, cmd)
                        }
                    }
                }
            } else if strings.HasSuffix(entry.Name(), ".md") {
                path := filepath.Join(searchPath, entry.Name())
                if cmd := c.loadFromFile(path, ""); cmd != nil {
                    commands = append(commands, cmd)
                }
            }
        }
    }

    return commands, nil
}

func (c *CustomCommandLoader) loadFromFile(path, namespace
        string) *CustomCommand {
    data, err := os.ReadFile(path)
    if err != nil {
        return nil
    }

    content := string(data)
    if !strings.HasPrefix(content, "---") {
        return nil
    }

    // Extract frontmatter
    endIdx := strings.Index(content[3:], "---")
    if endIdx < 0 {
        return nil
    }

    frontmatter := content[3 : endIdx+3]
    prompt := strings.TrimSpace(content[endIdx+6:])

    var cmd CustomCommand
    if err := yaml.Unmarshal([]byte(frontmatter), &cmd); err !=
        nil {
        return nil
    }

    cmd.Prompt = prompt
    cmd.Namespace = namespace
```

```
    // If no name, derive from filename
    if cmd.Name == "" {
        basename := filepath.Base(path)
        cmd.Name = strings.TrimSuffix(basename, ".md")
    }

    return &cmd
}

// Execute runs a custom command with given arguments
func (c *CustomCommand) Execute(args map[string]string) (string,
        error) {
    prompt := c.Prompt

    // Replace template variables
    for _, param := range c.Parameters {
        value, exists := args[param.Name]
        if !exists {
            value = param.Default
        }
        if param.Required && value == "" {
            return "", fmt.Errorf("required parameter missing:
        %s", param.Name)
        }
        placeholder := fmt.Sprintf("{{%s}}", param.Name)
        prompt = strings.ReplaceAll(prompt, placeholder, value)
    }

    return prompt, nil
}
```

Porting Complexity: Medium

- Checkpointing requires conversation state serialization
- Custom commands need template engine
- 4-tier modes are simple state machine

Anti-Bluff Test

```
# Test 1: Checkpoint
cat > test_conversation.txt << 'EOF'
User: Hello
AI: Hi there
EOF
helix checkpoint save --name "before-refactor"
# Expected: Checkpoint created with ID

helix checkpoint list
# Expected: Shows checkpoint with timestamp
```

```
helix checkpoint restore cp-12345
# Expected: Conversation restored to checkpoint state

# Test 2: Custom command
mkdir -p .helix/commands/
cat > .helix/commands/review.md << 'EOF'
---
name: review
description: Code review command
parameters:
  - name: file
    type: string
    required: true
---
Please review {{file}} for security issues, performance
        bottlenecks, and code style.
EOF
helix commands list
# Expected: "review" command listed

helix review --file main.go
# Expected: Custom prompt executed with file parameter
        substituted
```

Agent 18: Noi

Source: `cli_agents/noi` **Language**: TypeScript/Electron **License**: Unknown

Feature Inventory - Top 3 Unique Features

1. Multi-Service AI Hub - Single UI for ChatGPT, Claude, Gemini, Perplexity - Session isolation per service - Anonymous usage for some services (no login required)

2. Multi-Window Management - Tab-based interface for multiple AI services - Local-first data for history and prompts - Prompt management system

3. Built-in Terminal - Integrated terminal for local commands - Ollama integration via terminal - Multiple themes

HelixCode Integration

Feature	HelixCode Package	Integration Point
Multi-provider UI	`applications/desktop/`	Extend Fyne app with provider tabs
Session isolation	`internal/session/`	Per-provider session isolation

Feature	HelixCode Package	Integration Point
Prompt management	`internal/memory/`	Prompt library and templates

Porting Complexity: Medium

- Desktop app requires UI work
- Multi-provider switching already partially supported
- Prompt library is new feature

Anti-Bluff Test

```
# Desktop app verification - not CLI testable
```

Agent 19: Octogen

Source: `cli_agents/octogen` **Language**: Python **License**: Unknown

Feature Inventory - Top 3 Unique Features

1. Multi-Agent Code Generation - Multiple agents collaborate on code generation - Agent specialization: planner, coder, reviewer - Iterative improvement through agent feedback

2. AutoGen Integration - Based on Microsoft AutoGen framework - Conversational agent groups - Human-in-the-loop workflows

3. AgentChat Architecture - Coder + Executor + User Proxy agent triad - Code generation -> execution -> feedback loop - Visual workflow builder (AutoGen Studio)

HelixCode Integration

Feature	HelixCode Package	Integration Point
Agent triad	`internal/agent/types/`	New agent types: Coder, Executor, Proxy
AutoGen patterns	`internal/agent/swarm/`	Add AutoGen-style conversation
Visual builder	`applications/desktop/`	Extend with workflow canvas

Porting Complexity: High

- Multi-agent code generation is complex coordination
- AutoGen patterns need message routing infrastructure
- Visual builder requires significant UI work

Anti-Bluff Test

```
helix swarm create --pattern autogen --agents
        "coder,executor,reviewer"
# Expected: Creates 3 specialized agents

helix swarm run --task "Generate a REST API in Go"
# Expected: Coder generates, Executor runs, Reviewer checks
```

Agent 20: Ollama_Code

> **Source**: `cli_agents/ollama-code` **Language**: TypeScript (fork of Qwen Code) **License**: MIT

Feature Inventory - Top 3 Unique Features

1. Privacy-First Local Model Support - All processing happens locally via Ollama - Complete privacy, no data transmission - Offline capability once models downloaded

2. Codebase Understanding Beyond Context Windows - Query and edit large codebases - Smart context selection for local models - Optimized for smaller models (7B-14B parameters)

3. Workflow Automation - Automate PR handling and complex rebases - Operational task automation - Git workflow integration

HelixCode Integration

Feature	HelixCode Package	Integration Point
Ollama support	`internal/llm/`	Already supported - verify
Smart context	`internal/context/builder.go`	Enhance for limited context
Workflow automation	`internal/workflow/`	Extend with git workflows

Porting Complexity: Low

- Ollama already supported
- Smart context selection is enhancement
- Workflow automation exists

Anti-Bluff Test

```
helix llm providers | grep -i ollama
# Expected: Ollama listed

helix chat --provider ollama --model qwen2.5-coder:14b
# Expected: Local chat session
```

Agent 21: Postgres-MCP

Source: `cli_agents/postgres-mcp` **Language**: TypeScript **License**: MIT

Feature Inventory - Top 3 Unique Features

1. Full Schema Introspection - Primary keys, foreign keys, indexes, column types, constraints - Complete database structure understanding - Multi-schema support

2. Performance Analysis Tools - pg_stat_statements integration - Slow query identification - EXPLAIN plan analysis - Hypothetical index simulation

3. Database Health Monitoring - Buffer cache hit rates - Connection health - Index health (duplicate/unused/invalid) - Vacuum health - Replication lag detection

HelixCode Integration

Feature	HelixCode Package	Integration Point
Postgres tools	`internal/tools/database/`	New package - Postgres operations
MCP exposure	`internal/mcp/tools/database.go`	MCP tool definitions
Health monitoring	`internal/monitoring/`	Database health checks

Exact Code Changes

New File: `internal/tools/database/postgres.go`

```
package database

import (
    "context"
    "database/sql"
    "fmt"

    _ "github.com/lib/pq"
)

// PostgresTool provides database operations
type PostgresTool struct {
    db *sql.DB
}

func NewPostgresTool(connectionString string) (*PostgresTool,
        error) {
    db, err := sql.Open("postgres", connectionString)
    if err != nil {
        return nil, err
    }
```

```
    return &PostgresTool{db: db}, nil
}

// ListSchemas returns all database schemas
func (p *PostgresTool) ListSchemas(ctx context.Context)
        ([]string, error) {
    rows, err := p.db.QueryContext(ctx, `
        SELECT schema_name
        FROM information_schema.schemata
        WHERE schema_name NOT IN ('pg_catalog',
        'information_schema')
    `)
    if err != nil {
        return nil, err
    }
    defer rows.Close()

    var schemas []string
    for rows.Next() {
        var schema string
        if err := rows.Scan(&schema); err != nil {
            continue
        }
        schemas = append(schemas, schema)
    }
    return schemas, nil
}

// GetTableDetails returns full table structure
func (p *PostgresTool) GetTableDetails(ctx context.Context,
        schema, table string) (*TableInfo, error) {
    // Query columns
    columns, err := p.getColumns(ctx, schema, table)
    if err != nil {
        return nil, err
    }

    // Query constraints
    constraints, err := p.getConstraints(ctx, schema, table)
    if err != nil {
        return nil, err
    }

    // Query indexes
    indexes, err := p.getIndexes(ctx, schema, table)
    if err != nil {
        return nil, err
    }
```

```
    return &TableInfo{
        Schema:      schema,
        Name:        table,
        Columns:     columns,
        Constraints: constraints,
        Indexes:     indexes,
    }, nil
}

// ExecuteSQL runs a SQL query safely
func (p *PostgresTool) ExecuteSQL(ctx context.Context, query
        string, readonly bool) (*QueryResult, error) {
    if readonly {
        // Verify query is read-only
        if !isReadOnlyQuery(query) {
            return nil, fmt.Errorf("query is not read-only: %s",
        query)
        }
    }

    rows, err := p.db.QueryContext(ctx, query)
    if err != nil {
        return nil, err
    }
    defer rows.Close()

    return p.rowsToResult(rows)
}

// ExplainQuery returns execution plan
func (p *PostgresTool) ExplainQuery(ctx context.Context, query
        string) (string, error) {
    rows, err := p.db.QueryContext(ctx, "EXPLAIN (ANALYZE,
        BUFFERS, FORMAT JSON) "+query)
    if err != nil {
        return "", err
    }
    defer rows.Close()

    var plan string
    if rows.Next() {
        rows.Scan(&plan)
    }
    return plan, nil
}

// HealthCheck performs comprehensive health analysis
```

```
func (p *PostgresTool) HealthCheck(ctx context.Context)
        (*HealthReport, error) {
    report := &HealthReport{}

    // Buffer cache hit rate
    p.db.QueryRowContext(ctx, `
        SELECT round(blks_hit::numeric / (blks_hit + blks_read) *
        100, 2)
        FROM pg_stat_database
        WHERE datname = current_database()
    `).Scan(&report.BufferCacheHitRate)

    // Connection count
    p.db.QueryRowContext(ctx, `
        SELECT count(*) FROM pg_stat_activity
    `).Scan(&report.ActiveConnections)

    // Unused indexes
    rows, _ := p.db.QueryContext(ctx, `
        SELECT schemaname, tablename, indexname
        FROM pg_stat_user_indexes
        WHERE idx_scan = 0
        AND indexrelname NOT LIKE 'pg_toast%'
    `)
    defer rows.Close()

    for rows.Next() {
        var schema, table, index string
        rows.Scan(&schema, &table, &index)
        report.UnusedIndexes = append(report.UnusedIndexes,
            fmt.Sprintf("%s.%s.%s", schema, table, index))
    }

    return report, nil
}

type TableInfo struct {
    Schema      string
    Name        string
    Columns     []ColumnInfo
    Constraints []ConstraintInfo
    Indexes     []IndexInfo
}

type ColumnInfo struct {
    Name     string
    Type     string
    Nullable bool
```

```
    Default  string
}

type ConstraintInfo struct {
    Name     string
    Type     string // PRIMARY KEY, FOREIGN KEY, UNIQUE, CHECK
    Columns  []string
}

type IndexInfo struct {
    Name    string
    Columns []string
    Unique  bool
}

type QueryResult struct {
    Columns []string
    Rows    [][]interface{}
    Count   int
}

type HealthReport struct {
    BufferCacheHitRate float64
    ActiveConnections  int
    UnusedIndexes      []string
    // ... more metrics
}

func isReadOnlyQuery(query string) bool {
    upper := strings.ToUpper(strings.TrimSpace(query))
    // Simple check - production code needs proper SQL parser
    forbidden := []string{"INSERT", "UPDATE", "DELETE", "DROP",
        "CREATE", "ALTER", "TRUNCATE"}
    for _, f := range forbidden {
        if strings.Contains(upper, f) {
            return false
        }
    }
    return true
}

func (p *PostgresTool) rowsToResult(rows *sql.Rows)
        (*QueryResult, error) {
    // Implementation
    return nil, nil
}
```

```
func (p *PostgresTool) getColumns(ctx context.Context, schema,
        table string) ([]ColumnInfo, error) {
    // Implementation
    return nil, nil
}

func (p *PostgresTool) getConstraints(ctx context.Context,
        schema, table string) ([]ConstraintInfo, error) {
    // Implementation
    return nil, nil
}

func (p *PostgresTool) getIndexes(ctx context.Context, schema,
        table string) ([]IndexInfo, error) {
    // Implementation
    return nil, nil
}
```

Porting Complexity: Medium

- Postgres operations are well-understood
- SQL safety requires parser (or regex fallback)
- Health checks need comprehensive queries

Anti-Bluff Test

```
helix db connect "postgres://user:pass@localhost/db"
helix db schema list
# Expected: List of schemas

helix db table details --schema public --table users
# Expected: Full table structure with PK, FK, indexes

helix db query "SELECT * FROM users LIMIT 5"
# Expected: Query results in table format

helix db health
# Expected: Health report with cache hit rate, unused indexes
```

Agent 22: Qwen_Code

Source: `cli_agents/qwen-code` **Language**: TypeScript (fork of Gemini CLI)
License: Apache 2.0

Feature Inventory - Top 3 Unique Features

1. Massive Context Windows (1M tokens) - 256K native context, up to 1M with YaRN extrapolation - Handle entire codebases in single conversation - Full-stack automation capability

2. Qwen3-Coder Model Optimization - 480B parameter MoE (35B active) - Agentic RL training for tool use - 20K parallel environments for training - SWE-Bench competitive performance

3. Enhanced Parser for Qwen Models - Customized prompts and function calling protocols - Optimized structured output parsing - Alibaba Cloud / ModelScope endpoint support

HelixCode Integration

Feature	HelixCode Package	Integration Point
Qwen provider	`internal/llm/`	Already exists - verify 1M context
Enhanced parser	`internal/llm/parsing/`	New - model-specific output parsing
YaRN support	`internal/llm/`	Context extrapolation configuration

Porting Complexity: Low

- Qwen already supported in LLM factory
- Enhanced parser is provider enhancement
- 1M context is provider-level feature

Anti-Bluff Test

```
helix llm providers | grep -i qwen
# Expected: Qwen listed with 1M context models

helix chat --provider qwen --context-window 1000000
# Expected: Session starts with extended context
```

Agent 23: Shai

Source: `cli_agents/shai` **Language**: Rust **License**: MIT

Feature Inventory - Top 3 Unique Features

1. Shell Assistant Mode - Automatically suggests fixes when commands fail - Watches terminal for failed commands - Proposes corrections inline

2. OpenAI-Compatible HTTP Server - Run shai as service with SSE streaming - OpenAI-compatible APIs - Background agent support

3. SHAI.md Project Context - `SHAI.md` files for project-specific information - Similar to CLAUDE.md but for Shai - Load project context automatically

HelixCode Integration

Feature	HelixCode Package	Integration Point
Shell assistant	`internal/tools/shell/`	Add failure detection + suggestions

Feature	HelixCode Package	Integration Point
HTTP server mode	`cmd/server/main.go`	Already exists - extend with OpenAI compat
Project context	`internal/context/providers/`	Add SHAI.md provider

Porting Complexity: Low-Medium

- Shell failure detection needs process monitoring
- HTTP server already exists
- SHAI.md is similar to existing context providers

Anti-Bluff Test

```
# Terminal 1: Run helix server
helix server --openai-compatible

# Terminal 2: Curl the API
curl http://localhost:8080/v1/chat/completions \
  -H "Content-Type: application/json" \
  -d '{"model":"helix","messages":
        [{"role":"user","content":"Hello"}]}'
# Expected: OpenAI-compatible response
```

Agent 24: SnowCLI

Source: `cli_agents/snow-cli` **Language**: Python **License**: Unknown

Feature Inventory - Top 3 Unique Features

1. Snowflake Cortex Code CLI - Natural language to SQL conversion - Snowflake-native AI integration - SSO and programmatic token authentication

2. SnowConvert AI (scai) - Database migration to Snowflake - AI-powered code improvement - Automated conversion from SQL Server, Redshift

3. Data Warehouse Operations - Query data warehouse in plain English - Built-in table viewer - Session pause/resume

HelixCode Integration

Feature	HelixCode Package	Integration Point
Snowflake tools	`internal/tools/database/snowflake.go`	New - Snowflake operations
Natural language SQL	`internal/llm/`	LLM-powered SQL generation
Migration	`internal/workflow/`	Migration workflow templates

Porting Complexity: Medium

- Snowflake connector needs snowflake-connector-python equivalent
- Natural language to SQL is LLM integration
- Migration is workflow orchestration

Anti-Bluff Test

```
helix db connect --type snowflake --account myaccount --user me
helix db query "Show me average sales by region"
# Expected: Natural language converted to SQL and executed
```

Agent 25: Stark-Kitty-Kiro-Cli

Source: `cli_agents/kiro-cli` **Language**: TypeScript **License**: Unknown

Feature Inventory - Top 3 Unique Features

1. Terminal UI Adaptation - Detects terminal capabilities automatically - Progress indicator in terminal title - Clickable hyperlinks in supported terminals - Theme detection (dark/light) - 256-color fallback

2. Context Management with /context - `/context show` - view context breakdown with per-file token usage - `/context add "src/**/*.ts"` - add by glob - `/context remove src/app.js` - remove specific file - `/context clear` - remove all rules

3. Editor Integration - `/editor` opens external editor for composing prompts - `/reply` opens editor with last assistant message quoted - Multi-line input support (Shift+Enter, Ctrl+J, Alt+Enter)

HelixCode Integration

Feature	HelixCode Package	Integration Point
Terminal adaptation	`applications/terminal_ui/`	Enhance with capability detection
Context commands	`internal/context/`	Add /context slash commands
External editor	`internal/editor/external.go`	New - editor launch

Exact Code Changes

New File: `internal/editor/external.go`

```
package editor

import (
    "fmt"
```

```
    "os"
    "os/exec"
    "path/filepath"
    "strings"
)

// ExternalEditor launches the user's preferred editor
type ExternalEditor struct {
    editor string
}

func NewExternalEditor() *ExternalEditor {
    editor := os.Getenv("EDITOR")
    if editor == "" {
        editor = os.Getenv("VISUAL")
    }
    if editor == "" {
        // Try common editors
        for _, candidate := range []string{"vim", "vi", "nano",
        "emacs", "code"} {
            if _, err := exec.LookPath(candidate); err == nil {
                editor = candidate
                break
            }
        }
    }
    return &ExternalEditor{editor: editor}
}

// Compose opens editor for writing a prompt, returns content
func (e *ExternalEditor) Compose(initialContent string) (string,
        error) {
    tmpFile, err := os.CreateTemp("", "helix-compose-*.md")
    if err != nil {
        return "", err
    }
    defer os.Remove(tmpFile.Name())

    if initialContent != "" {
        if _, err := tmpFile.WriteString(initialContent); err !=
        nil {
            tmpFile.Close()
            return "", err
        }
    }
    tmpFile.Close()

    // Launch editor
```

```
    cmd := exec.Command(e.editor, tmpFile.Name())
    cmd.Stdin = os.Stdin
    cmd.Stdout = os.Stdout
    cmd.Stderr = os.Stderr

    if err := cmd.Run(); err != nil {
        return "", fmt.Errorf("editor failed: %w", err)
    }

    // Read result
    content, err := os.ReadFile(tmpFile.Name())
    if err != nil {
        return "", err
    }

    return string(content), nil
}

// Reply opens editor with quoted assistant message
func (e *ExternalEditor) Reply(assistantMessage string) (string,
        error) {
    quoted := "> " + strings.ReplaceAll(assistantMessage, "\n",
        "\n> ")
    initial := fmt.Sprintf("\n\n%s\n\n", quoted)
    return e.Compose(initial)
}
```

New File: `internal/commands/context.go`

```
package commands

import (
    "context"
    "fmt"
    "strings"
)

// ContextCommand manages session context
var ContextCmd = &CommandDefinition{
    Name:        "context",
    Description: "Manage session context",
    Subcommands: map[string]CommandHandler{
        "show":   contextShowHandler,
        "add":    contextAddHandler,
        "remove": contextRemoveHandler,
        "clear": contextClearHandler,
    },
}
```

```
func contextShowHandler(ctx context.Context, args []string)
        (string, error) {
    // Show context breakdown with per-file token usage
    session := GetCurrentSession(ctx)
    items := session.Context.Items()

    var output strings.Builder
    output.WriteString("Context Breakdown:\n")
    output.WriteString(fmt.Sprintf("Total tokens: %d\n\n",
        session.Context.TotalTokens()))

    for _, item := range items {
        output.WriteString(fmt.Sprintf("  %s (%d tokens): %s\n",
            item.Type, item.Tokens, item.Key))
    }

    return output.String(), nil
}

func contextAddHandler(ctx context.Context, args []string)
        (string, error) {
    if len(args) < 1 {
        return "", fmt.Errorf("pattern required")
    }
    pattern := args[0]

    session := GetCurrentSession(ctx)
    files, err := globFiles(pattern)
    if err != nil {
        return "", err
    }

    for _, file := range files {
        session.Context.AddFile(file)
    }

    return fmt.Sprintf("Added %d files matching %s", len(files),
        pattern), nil
}

func contextRemoveHandler(ctx context.Context, args []string)
        (string, error) {
    if len(args) < 1 {
        return "", fmt.Errorf("file path required")
    }
    session := GetCurrentSession(ctx)
    session.Context.Remove(args[0])
    return fmt.Sprintf("Removed %s from context", args[0]), nil
```

```
}

func contextClearHandler(ctx context.Context, args []string)
        (string, error) {
    session := GetCurrentSession(ctx)
    session.Context.Clear()
    return "Context cleared", nil
}
```

Porting Complexity: Low

- External editor is simple exec call
- Context commands leverage existing context manager
- Terminal adaptation is UI enhancement

Anti-Bluff Test

```
# Test 1: External editor
export EDITOR=vim
helix editor compose
# Expected: vim opens with temp file, content returned on save

# Test 2: Context show
helix context show
# Expected: Shows files in context with token counts

# Test 3: Context add by glob
helix context add "src/**/*.go"
# Expected: All matching Go files added to context

# Test 4: Context remove
helix context remove src/main.go
# Expected: File removed from context
```

Agent 26: Superset

Source: `cli_agents/superset` **Language**: Python **License**: Apache 2.0

Feature Inventory - Top 3 Unique Features

1. Data Visualization CLI - Apache Superset integration for data viz - Chart SQL access: get compiled SQL behind any chart - Chart data export (JSON, CSV)

2. Asset Management - Backup/restore charts, dashboards, datasets - Synchronize assets across instances - Jinja2 templating for customization - Git-ready YAML-based assets

3. Agent-Optimized Output - `--json` for AI agents and automation - `--csv` for direct data export - `--porcelain` for machine-readable output - Server-side search across entities

HelixCode Integration

Feature	HelixCode Package	Integration Point
Superset tools	`internal/tools/superset/`	New - BI/data viz operations
Asset sync	`internal/workflow/`	Asset synchronization workflows
Data export	`internal/tools/database/`	Query result formatting

Porting Complexity: Medium

- Superset API integration
- Data visualization requires chart rendering
- Asset sync is workflow orchestration

Anti-Bluff Test

```
helix superset connect --url https://superset.company.com
helix superset chart sql 3628
# Expected: Shows compiled SQL for chart

helix superset chart data 3628 --json
# Expected: Chart data as JSON
```

Agent 27: TaskWeaver

> **Source**: `cli_agents/taskweaver` **Language**: Python **License**: MIT (Microsoft)

Feature Inventory - Top 3 Unique Features

1. Code-First Agent Framework - Converts user requests into Python programs - Stateful execution like Jupyter Notebook - Rich data structures: DataFrames, ndarrays in memory - Plugin system for custom algorithms

2. Domain Adaptation - Custom plugins with Python + schema - Example-based customization (YAML) - Planning examples + code generation examples - Experience feature with static/dynamic selection

3. Multi-Agent Architecture - Planner: task decomposition and management - Code Interpreter (CI): code execution - Code Generator (CG): code creation - Code Executor (CE): safe execution - Recepta: reasoning power (experimental) - Shared memory between roles

HelixCode Integration

Feature	HelixCode Package	Integration Point
Code-first execution	`internal/tools/sandbox/python.go`	New - Python sandbox
Plugin system	`internal/skills/`	Extend with Python plugins

Feature	HelixCode Package	Integration Point
Multi-agent roles	`internal/agent/types/`	New roles: Planner, CI, CG, CE
Shared memory	`internal/memory/shared.go`	New - cross-role memory

Exact Code Changes

New File: `internal/tools/sandbox/python.go`

```go
package sandbox

import (
    "context"
    "fmt"
    "os"
    "os/exec"
    "path/filepath"
)

// PythonSandbox executes Python code safely
type PythonSandbox struct {
    workDir     string
    venvPath    string
    timeout     int
    maxMemoryMB int
    allowedImports []string
}

func NewPythonSandbox(workDir string) (*PythonSandbox, error) {
    absPath, err := filepath.Abs(workDir)
    if err != nil {
        return nil, err
    }

    return &PythonSandbox{
        workDir:     absPath,
        timeout:     30,
        maxMemoryMB: 512,
        allowedImports: []string{
            "pandas", "numpy", "matplotlib", "seaborn",
            "sklearn", "requests", "json", "csv",
        },
    }, nil
}

// Execute runs Python code in sandbox
```

```
func (ps *PythonSandbox) Execute(ctx context.Context, code
        string) (*ExecutionResult, error) {
    // Write code to temp file
    tmpFile := filepath.Join(ps.workDir,
        fmt.Sprintf("script_%d.py", time.Now().Unix()))
    if err := os.WriteFile(tmpFile, []byte(code), 0644); err !=
        nil {
        return nil, err
    }
    defer os.Remove(tmpFile)

    // Build command with restrictions
    args := []string{
        "python3",
        tmpFile,
    }

    // Use timeout, ulimit for memory
    cmd := exec.CommandContext(ctx, args[0], args[1:]...)
    cmd.Dir = ps.workDir
    cmd.Env = ps.buildEnv()

    // Capture output
    output, err := cmd.CombinedOutput()

    result := &ExecutionResult{
        Output:   string(output),
        ExitCode: 0,
    }

    if err != nil {
        if exitErr, ok := err.(*exec.ExitError); ok {
            result.ExitCode = exitErr.ExitCode()
        } else {
            result.ExitCode = -1
            result.Error = err.Error()
        }
    }

    return result, nil
}

// ExecuteWithDataFrames runs code with DataFrame inputs/outputs
func (ps *PythonSandbox) ExecuteWithDataFrames(ctx
        context.Context, code string, inputs map[string][]byte)
        (*DataFrameResult, error) {
    // Serialize inputs as parquet
    // Execute code
```

```
    // Deserialize outputs as parquet
    // ... implementation
    return nil, nil
}

func (ps *PythonSandbox) buildEnv() []string {
    env := os.Environ()
    // Restrict PYTHONPATH
    // Set memory limits
    // Disable network if needed
    return env
}

type ExecutionResult struct {
    Output   string
    ExitCode int
    Error    string
}

type DataFrameResult struct {
    ExecutionResult
    DataFrames map[string][]byte // parquet-encoded DataFrames
}
```

New File: `internal/memory/shared.go`

```
package memory

import (
    "context"
    "fmt"
    "sync"
    "time"
)

// SharedMemory stores information shared between agent roles
type SharedMemory struct {
    mu      sync.RWMutex
    entries map[string]*SharedEntry
}

type SharedEntry struct {
    Key       string
    Value     interface{}
    Role      string      // Which role created this
    CreatedAt time.Time
    AccessLog []AccessRecord
}
```

```
type AccessRecord struct {
    Role      string
    Action    string // read, write, delete
    Timestamp time.Time
}

func NewSharedMemory() *SharedMemory {
    return &SharedMemory{
        entries: make(map[string]*SharedEntry),
    }
}

// Write stores a value, accessible to all roles
func (sm *SharedMemory) Write(ctx context.Context, key string,
        value interface{}, role string) error {
    sm.mu.Lock()
    defer sm.mu.Unlock()

    entry := &SharedEntry{
        Key:       key,
        Value:     value,
        Role:      role,
        CreatedAt: time.Now(),
        AccessLog: []AccessRecord{
            {Role: role, Action: "write", Timestamp: time.Now()},
        },
    }

    sm.entries[key] = entry
    return nil
}

// Read retrieves a value and logs access
func (sm *SharedMemory) Read(ctx context.Context, key string,
        role string) (interface{}, error) {
    sm.mu.Lock()
    defer sm.mu.Unlock()

    entry, exists := sm.entries[key]
    if !exists {
        return nil, fmt.Errorf("key not found: %s", key)
    }

    entry.AccessLog = append(entry.AccessLog, AccessRecord{
        Role:      role,
        Action:    "read",
        Timestamp: time.Now(),
    })
```

```
    return entry.Value, nil
}

// List returns all keys
func (sm *SharedMemory) List() []string {
    sm.mu.RLock()
    defer sm.mu.RUnlock()

    keys := make([]string, 0, len(sm.entries))
    for k := range sm.entries {
        keys = append(keys, k)
    }
    return keys
}

// Delete removes an entry
func (sm *SharedMemory) Delete(ctx context.Context, key string,
        role string) error {
    sm.mu.Lock()
    defer sm.mu.Unlock()

    if _, exists := sm.entries[key]; !exists {
        return fmt.Errorf("key not found: %s", key)
    }

    delete(sm.entries, key)
    return nil
}
```

New File: `internal/agent/types/planner_agent.go`

```
package types

import (
    "context"
    "fmt"
)

// PlannerAgent decomposes tasks into subtasks
type PlannerAgent struct {
    BaseAgent
    sharedMemory *memory.SharedMemory
}

func NewPlannerAgent(sharedMemory *memory.SharedMemory)
        *PlannerAgent {
    return &PlannerAgent{
        BaseAgent:    NewBaseAgent("planner", AgentTypePlanning),
```

```
        sharedMemory: sharedMemory,
    }
}

func (pa *PlannerAgent) Execute(ctx context.Context, task
        *task.Task) (*task.Result, error) {
    // Analyze task
    // Decompose into subtasks
    // Store plan in shared memory
    // Coordinate with CI, CG, CE agents

    plan := &TaskPlan{
        OriginalTask: task,
        Steps: []PlanStep{
            {ID: "1", Action: "analyze", Description: "Analyze
        requirements"},
            {ID: "2", Action: "generate", Description: "Generate
        code"},
            {ID: "3", Action: "execute", Description: "Execute
        and verify"},
        },
    }

    pa.sharedMemory.Write(ctx, "current_plan", plan, "planner")

    return &task.Result{
        Success: true,
        Data:    plan,
    }, nil
}

type TaskPlan struct {
    OriginalTask *task.Task
    Steps        []PlanStep
}

type PlanStep struct {
    ID          string
    Action      string
    Description string
    Status      string
    Result      interface{}
}
```

Porting Complexity: High

- Python sandbox needs security (code injection prevention)
- Multi-agent roles require coordination protocol
- Shared memory needs access control and versioning

Anti-Bluff Test

```
# Test 1: Python sandbox
helix sandbox python "import pandas as pd; df =
        pd.DataFrame({'a': [1,2,3]}); print(df)"
# Expected: DataFrame printed

# Test 2: Shared memory
helix memory shared write --key "plan" --value '{"steps": 3}' --
        role planner
helix memory shared read --key "plan" --role executor
# Expected: Value retrieved, access logged

# Test 3: Planner agent
helix agent run --type planner --task "Analyze sales data"
# Expected: Task decomposed into steps, stored in shared memory
```

Agent 28: Warp

Source: `cli_agents/warp` **Language**: Rust **License**: Proprietary

Feature Inventory - Top 3 Unique Features

1. Terminal-First Agent UX - Full terminal replacement (not bolted-on chat panel) - Shell as primary UI surface - Commands, output, and agent reasoning share same surface

2. Agent Mode with Command Loop - Agent proposes commands, reads output, iterates - Real toolchain context (files, env vars, processes) - Command-first verbs: run, retry, pipe, rollback

3. Warp Drive - Team Knowledge Sharing - Shared repository of commands, notebooks, prompts - Semantic search for team knowledge - Saved prompts as institutional memory

HelixCode Integration

Feature	HelixCode Package	Integration Point
Terminal agent UX	`applications/terminal_ui/`	Enhance with command-aware UI
Agent mode	`internal/session/modes.go`	Add terminal-native agent mode
Team knowledge	`internal/memory/team/`	New - shared prompt/command library

Porting Complexity: High

- Terminal replacement requires significant TUI work
- Command-aware agent needs shell integration
- Team knowledge needs sharing infrastructure

Anti-Bluff Test

```
# Requires terminal UI implementation
helix --mode agent
# Expected: Terminal becomes agent-aware, proposals inline
```

Agent 29: vtcode

Source: `cli_agents/vtcode` **Language**: Swift **License**: MIT

Feature Inventory - Top 3 Unique Features

1. Security-First Architecture - Tree-sitter-bash command validation - Execution policy with per-command argument validation - macOS Seatbelt + Linux Landlock +seccomp sandboxing (similar to Codex) - Per-command arg validation (regex patterns) - Flat command spaces with limited commandability

2. AI-Driven Query Auto-Completion - Rich inline suggestions while typing - Shell command completion with AI assistance - Real-time prediction of next commands

3. Swift-Optimized Terminal - Native macOS performance - SwiftUI integration - macOS-specific optimizations

HelixCode Integration

Feature	HelixCode Package	Integration Point
Command validation	`internal/tools/shell/security.go`	New - per-command validation
Sandbox policies	`internal/sandbox/`	Extend with Landlock + Seatbelt
Inline completion	`applications/terminal_ui/`	Rich inline suggestions

Exact Code Changes

New File: `internal/tools/shell/security.go`

```
package shell

import (
    "context"
    "fmt"
    "regexp"
    "strings"
)

// CommandValidator validates shell commands for safety
type CommandValidator struct {
```

```
    policies map[string]*CommandPolicy
}

// CommandPolicy defines what a command is allowed to do
type CommandPolicy struct {
    Command       string
    AllowedArgs   []AllowedArg
    ForbiddenArgs []string
    MaxArgCount   int
    AllowedDirs   []string
    MaxFileSize   int64
    ReadOnly      bool
}

type AllowedArg struct {
    Pattern string // regex
    Values  []string // exact values
}

func NewCommandValidator() *CommandValidator {
    cv := &CommandValidator{
        policies: make(map[string]*CommandPolicy),
    }

    // Default policies
    cv.policies["cat"] = &CommandPolicy{
        Command:       "cat",
        AllowedArgs:   []AllowedArg{{Pattern: `^[\w\-\.\/]+$`}},
        ForbiddenArgs: []string{"-", "--"},
        MaxArgCount:   10,
        ReadOnly:      true,
    }

    cv.policies["ls"] = &CommandPolicy{
        Command:       "ls",
        AllowedArgs:   []AllowedArg{{Pattern: `^-[a-zA-Z]+$`}},
        MaxArgCount:   5,
        ReadOnly:      true,
    }

    cv.policies["rm"] = &CommandPolicy{
        Command:       "rm",
        AllowedArgs:   []AllowedArg{{Pattern: `^-[a-zA-Z]+$`}},
        ForbiddenArgs: []string{"-rf", "-fr", "-f"},
        MaxArgCount:   3,
        ReadOnly:      false,
    }
```

```
    cv.policies["git"] = &CommandPolicy{
        Command:       "git",
        AllowedArgs:   []AllowedArg{{Values: []string{"status",
        "log", "diff", "branch", "remote", "show", "blame"}}},
        MaxArgCount:   10,
        ReadOnly:      true, // only safe git ops
    }

    return cv
}

// Validate checks if a command is safe to execute
func (cv *CommandValidator) Validate(command string, args
        []string) error {
    policy, exists := cv.policies[command]
    if !exists {
        return fmt.Errorf("command not in allowlist: %s",
        command)
    }

    if len(args) > policy.MaxArgCount {
        return fmt.Errorf("too many arguments for %s: %d > %d",
        command, len(args), policy.MaxArgCount)
    }

    for _, arg := range args {
        // Check forbidden args
        for _, forbidden := range policy.ForbiddenArgs {
            if strings.Contains(arg, forbidden) {
                return fmt.Errorf("forbidden argument in %s:
        %s", command, forbidden)
            }
        }

        // Check allowed patterns
        valid := false
        for _, allowed := range policy.AllowedArgs {
            if len(allowed.Values) > 0 {
                for _, v := range allowed.Values {
                    if arg == v || strings.HasPrefix(arg, v+"=")
        {
                        valid = true
                        break
                    }
                }
            }
            if allowed.Pattern != "" {
```

```
                matched, _ :=
            regexp.MatchString(allowed.Pattern, arg)
                if matched {
                    valid = true
                }
            }
        }
        if !valid && len(policy.AllowedArgs) > 0 {
            return fmt.Errorf("argument not allowed for %s: %s",
        command, arg)
        }
    }

    return nil
}

// RegisterPolicy adds a custom command policy
func (cv *CommandValidator) RegisterPolicy(policy
        *CommandPolicy) {
    cv.policies[policy.Command] = policy
}

// LoadPoliciesFromFile loads policies from YAML/JSON
func (cv *CommandValidator) LoadPoliciesFromFile(path string)
        error {
    // Parse and register policies
    return nil
}
```

New File: `internal/sandbox/landlock_linux.go`

```
//go:build linux
// +build linux

package sandbox

import (
    "fmt"
    "os/exec"
    "syscall"
)

// LandlockSandbox uses Linux Landlock LFS for filesystem
        sandboxing
type LandlockSandbox struct {
    allowedPaths []string
    readOnlyPaths []string
    readWritePaths []string
}
```

```go
func NewLandlockSandbox() *LandlockSandbox {
    return &LandlockSandbox{}
}

// AllowPath grants read access to a path
func (ls *LandlockSandbox) AllowPath(path string) {
    ls.allowedPaths = append(ls.allowedPaths, path)
}

// AllowReadWrite grants read-write access to a path
func (ls *LandlockSandbox) AllowReadWrite(path string) {
    ls.readWritePaths = append(ls.readWritePaths, path)
}

// Apply configures Landlock for the calling process
func (ls *LandlockSandbox) Apply() error {
    // Landlock ABI version check
    abi, err := syscall.LandlockCreateRuleset(0, 0)
    if err != nil {
        return fmt.Errorf("landlock not supported: %w", err)
    }
    syscall.Close(abi)

    // Create ruleset with FS-related access rights
    // Add rules for allowed paths
    // Enforce ruleset
    // ... implementation

    return nil
}

// WrapCommand wraps an exec.Cmd with Landlock
func (ls *LandlockSandbox) WrapCommand(cmd *exec.Cmd) *exec.Cmd {
    // Use landlock_wrap or LD_PRELOAD
    // ... implementation
    return cmd
}
```

Porting Complexity: High

- Security sandboxing needs OS-specific implementations
- Command validation requires comprehensive policy definitions
- Landlock is Linux-only, need macOS Seatbelt equivalent

Anti-Bluff Test

```
# Test 1: Command validation
helix shell validate "rm -rf /"
```

```
# Expected: FAIL - forbidden argument -rf

helix shell validate "ls -la src/"
# Expected: PASS

# Test 2: Policy loading
cat > .helix/shell-policies.yml << 'EOF'
policies:
  - command: custom-tool
    allowed_args:
      - pattern: '^[a-zA-Z0-9]+$'
    max_arg_count: 5
    read_only: true
EOF
helix shell policies load .helix/shell-policies.yml
# Expected: Custom policy loaded

# Test 3: Sandbox execution (Linux)
helix shell exec --sandbox --allow-read /tmp --allow-readwrite .
        "ls /tmp"
# Expected: Command runs with Landlock restrictions
```

Agent 30: gptme

Source: `cli_agents/gptme` **Language**: Python **License**: MIT

Feature Inventory - Top 3 Unique Features

1. Self-Programming Assistant - Programs itself (modifies own source code) - Full computer control (terminal + file system + browser + Python) - Extensible through “tools” (CLI, Python, Browser, Vision)

2. Multi-Model Support - Claude, GPT-4o, OpenAI, Anthropic, OpenRouter - Local models via Ollama/llm - Assistant message with nested tool calls (blocks model-generated output) - Each block starts with `# Assistant (` block

3. Extensible Tool System - Bash shell tool with multi-line support - Python REPL with session persistence - Browser automation via Playwright - Vision: screenshot + upload + OCR - Custom tool registration

HelixCode Integration

Feature	HelixCode Package	Integration Point
Self-programming	`internal/agent/self_modify.go`	New - agent self-modification
Nested tool calls	`internal/llm/parsing/`	Multi-level tool call parsing
Multi-line shell	`internal/tools/shell/`	Already exists - verify multi-line

Feature	HelixCode Package	Integration Point
Python REPL	`internal/tools/sandbox/`	New - persistent Python session

Porting Complexity: Medium

- Self-programming is architecturally interesting
- Multi-model already supported
- Tool system already exists

Anti-Bluff Test

```
# Test 1: Multi-model
helix llm switch --provider openrouter --model anthropic/claude-
        sonnet-4
# Expected: Switches active model

# Test 2: Nested tool calls
# Agent responds with:
# ```python
# print("Hello from Python")
# ```
# Expected: Python block detected and executed

# Test 3: Self-modification (with confirmation)
helix config modify "Add gemini provider"
# Expected: Helix modifies its own config with user approval
```

---

4. Quick Wins

Quick wins are features that can be ported in <1 day with minimal architectural changes:

#	Feature	Source Agent	HelixCode Package	Effort
1	**Todo Manager**	Mistral Code	`internal/tools/todo.go`	2h
2	**External Editor**	Stark-Kitty-Kiro-Cli	`internal/editor/external.go`	2h
3	**Checkpoint System**	Nanocoder	`internal/memory/checkpoint.go`	4h
4	**Custom Commands**	Nanocoder	`internal/commands/custom.go`	4h
5	**Profile System**	Bridle	`cmd/helix/profile.go`	4h
6	**Context Commands**	Stark-Kitty-Kiro-Cli	`internal/commands/context.go`	2h
7		DeepSeek CLI	`internal/llm/`	1h

#	Feature	Source Agent	HelixCode Package	Effort
	DeepSeek Provider Verify			
8	**Ollama Context Optimized**	Ollama Code	`internal/context/builder.go`	3h
9	**Get-Shit-Done Focus Mode**	Get-Shit-Done	`internal/session/`	2h
10	**Gemini Streaming Verify**	Gemini CLI	`internal/llm/streaming/`	1h
11	**Qwen Enhanced Parser**	Qwen Code	`internal/llm/parsing/`	3h
12	**GitMCP Tools**	GitMCP	`internal/mcp/tools/git.go`	3h
13	**Spec Management**	GitHub-Spec-Kit	`internal/specs/`	3h
14	**Postgres Health Checks**	Postgres-MCP	`internal/tools/database/postgres.go`	3h
15	**Prompt Management**	Noi	`internal/memory/prompts.go`	2h

Total Quick Win Time: ~40 hours (~5 person-days)

---

5. Game Changers

Game changers are features that significantly improve HelixCode's capabilities:

#	Feature	Source Agent	Impact	Complexity
1	**SKILL.md Progressive Disclosure**	Claude-Code-Plugins	Enables portable skills ecosystem	High
2	**Agent Teams with Mailbox**	Claude-Code-Plugins	True multi-agent collaboration	High
3	**Worktree Isolation**	Emdash	Parallel agents without conflicts	High
4	**Adversary Reviewer**	Codename_Goose		Medium

#	Feature	Source Agent	Impact	Complexity
			Security-first agent execution	
5	**Python Sandbox with DataFrames**	TaskWeaver	Data science agent capabilities	High
6	**Command Validation + Sandboxing**	vtcode	Safe agent execution	High
7	**Multi-Agent Role System**	TaskWeaver	Specialized agent roles	High
8	**Recipe System**	Codename_Goose	Reusable workflow automation	Medium
9	**Plan Mode with Shift+Tab**	GitHub-Copilot-CLI	Controlled agent execution	Medium
10	**Terminal-Native Agent UX**	Warp	Revolutionary CLI interaction	High
11	**Issue Tracker Integration**	Emdash	Enterprise workflow integration	Medium
12	**Custom Agent Definitions**	GitHub-Copilot-CLI	Domain-specific agents	Medium
13	**AutoGen Pattern Integration**	Octogen	Agent group collaboration	High
14	**Multi-Model Switching**	gptme	Dynamic provider selection	Low
15	**Superset BI Integration**	Superset	Business intelligence tools	Medium

---

6. Integration Matrix

Feature-to-Package Mapping

Feature	Package	File(s)	New/Modify
SKILL.md registry	`internal/skills/`	`registry.go`, `loader.go`, `watcher.go`	NEW

Feature	Package	File(s)	New/Modify
Plugin marketplace	`cmd/helix/`	`plugin.go`	NEW
Agent teams	`internal/agent/swarm/`	`coordinator.go` (extend)	MODIFY
Cross-harness translator	`internal/config/harness/`	`translator.go`, `matrix.go`	NEW
Profile system	`cmd/helix/`	`profile.go`	NEW
MCP Apps	`internal/mcp/apps/`	`renderer.go`, `ui.go`	NEW
Recipe system	`internal/workflow/recipes/`	`recipe.go`, `executor.go`	NEW
Adversary reviewer	`internal/agent/security/`	`adversary.go`	NEW
Worktree manager	`internal/git/`	`worktree.go`	NEW
Issue connectors	`internal/integrations/issues/`	`connector.go`, `github.go`, `jira.go`, `linear.go`	NEW
Plan mode	`internal/session/`	`modes.go`	MODIFY
Context compaction	`internal/llm/compression/`	`auto_compact.go`	MODIFY
Custom agents	`internal/agent/custom/`	`loader.go`	NEW
Todo manager	`internal/tools/`	`todo.go`	NEW
Checkpoint system	`internal/memory/`	`checkpoint.go`	NEW
Custom commands	`internal/commands/`	`custom.go`	NEW
Terminal adaptation	`applications/terminal_ui/`	`terminal.go` (extend)	MODIFY
Context commands	`internal/commands/`	`context.go`	NEW
External editor	`internal/editor/`	`external.go`	NEW
Python sandbox	`internal/tools/sandbox/`	`python.go`	NEW

Feature	Package	File(s)	New/Modify
Shared memory	`internal/memory/`	`shared.go`	NEW
Planner agent	`internal/agent/types/`	`planner_agent.go`, `code_interpreter.go`, `code_generator.go`, `code_executor.go`	NEW
Command validator	`internal/tools/shell/`	`security.go`	NEW
Landlock sandbox	`internal/sandbox/`	`landlock_linux.go`, `seatbelt_darwin.go`	NEW
Self-modification	`internal/agent/`	`self_modify.go`	NEW
Superset tools	`internal/tools/superset/`	`client.go`, `chart.go`, `dashboard.go`	NEW
Snowflake tools	`internal/tools/database/`	`snowflake.go`	NEW
Postgres tools	`internal/tools/database/`	`postgres.go`	NEW
Git MCP tools	`internal/mcp/tools/`	`git.go`, `github.go`	NEW
Focus mode	`internal/session/`	`focus.go`	NEW
Team knowledge	`internal/memory/team/`	`library.go`, `search.go`	NEW
DataFrame handling	`internal/tools/sandbox/`	`dataframe.go`	NEW
Multi-line shell	`internal/tools/shell/`	`multiline.go` (extend)	MODIFY
Nested tool parsing	`internal/llm/parsing/`	`nested.go`	NEW
Enhanced parser	`internal/llm/parsing/`	`model_specific.go`	NEW
Prompt library	`internal/memory/`	`prompts.go`	NEW
SHAI.md provider	`internal/context/providers/`	`shai_provider.go`	NEW
Visual workflows	`applications/desktop/`	`workflow_canvas.go` (extend)	MODIFY

Feature	Package	File(s)	New/Modify
Trigger system	`internal/workflow/triggers/`	`event_trigger.go`, `schedule_trigger.go`	NEW
Data blocks	`internal/workflow/blocks/`	`transform.go`, `filter.go`, `aggregate.go`	NEW
Completion API	`api/openapi.yaml`	`completion` endpoints	MODIFY
OpenAI-compatible	`cmd/server/`	`openai_compat.go`	NEW
Mobile automation	`internal/tools/mobile/`	`adb.go`, `uiautomator.go`	NEW
Mobile vision	`internal/llm/vision/`	`ui_analysis.go` (extend)	MODIFY
Multi-agent patterns	`internal/agent/swarm/`	`patterns.go`	NEW
Agent communication	`internal/agent/comm/`	`messaging.go`, `routing.go`	NEW
Verification	`internal/agent/verifier/`	`cross_agent.go`	NEW
Spec management	`internal/specs/`	`spec.go`, `loader.go`	NEW
FauxPilot local	`internal/llm/local/`	`fauxpilot.go`	NEW
Yolo mode	`internal/session/`	`yolo_mode.go` (extend modes)	MODIFY
Agent mode UX	`applications/terminal_ui/`	`agent_mode.go` (extend)	MODIFY
Team knowledge sharing	`internal/memory/team/`	`drive.go` (extend)	MODIFY

---

7. Implementation Schedule

Phase 1: Quick Wins (Week 1)

- [ ] Todo Manager (Mistral Code)
- [ ] External Editor (Kiro CLI)
- [ ] Context Commands (Kiro CLI)
- [ ]

- [ ] Profile System (Bridle)
- [ ] Custom Commands (Nanocoder)
- [ ] Checkpoint System (Nanocoder)
- [ ] GitMCP Tools (GitMCP)
- [ ] Postgres Health Checks (Postgres-MCP)
- [ ] Focus Mode (Get-Shit-Done)

Phase 2: Core Architecture (Weeks 2-3)

- [ ] SKILL.md Registry (Claude Plugins)
- [ ] Plan Mode + Agent Modes (Copilot CLI + Mistral Code)
- [ ] Custom Agent Loader (Copilot CLI)
- [ ] Command Validator (vtcode)
- [ ] Recipe System (Goose)
- [ ] Worktree Manager (Emdash)
- [ ] Shared Memory (TaskWeaver)
- [ ] Multi-Agent Roles (TaskWeaver)

Phase 3: Advanced Features (Weeks 4-5)

- [ ] Agent Teams with Mailbox (Claude Plugins)
- [ ] Cross-Harness Translator (Bridle)
- [ ] Adversary Reviewer (Goose)
- [ ] Python Sandbox (TaskWeaver)
- [ ] Terminal-Native Agent Mode (Warp)
- [ ] Team Knowledge Sharing (Warp)
- [ ] Issue Connectors (Emdash)

Phase 4: Specialized Integrations (Week 6)

- [ ] Database Tools (Postgres + Snowflake)
- [ ] Superset BI Integration
- [ ] MCP Apps Renderer (Goose)
- [ ] Security Sandboxing (vtcode)
- [ ] Self-Modification Agent (gptme)
- [ ] AutoGen Patterns (Octogen)
- [ ] Visual Workflows (Conduit)

---

8. Anti-Bluff Test Framework

End-to-End Test Suite

```
#!/bin/bash
# run_anti_bluff_tests.sh
# Comprehensive test suite for all ported features

set -e

echo "=== ANTI-BLUFF TEST SUITE ==="
echo ""

# ---- Quick Wins ----
echo "--- Quick Wins ---"

# Test 1: Todo Manager
helix todo add "Test task"
helix todo list | grep "Test task" && echo "PASS: Todo Manager"
        || echo "FAIL: Todo Manager"

# Test 2: External Editor
export EDITOR=cat
result=$(echo "Test content" | helix editor compose)
[[ "$result" == *"Test content"* ]] && echo "PASS: External
        Editor" || echo "FAIL: External Editor"

# Test 3: Context Commands
helix context add "*.go"
```

```
helix context show | grep "go" && echo "PASS: Context" || echo
        "FAIL: Context"

# Test 4: Profile System
helix profile create test --from-current
helix profile list | grep test && echo "PASS: Profile" || echo
        "FAIL: Profile"

# Test 5: Custom Commands
mkdir -p .helix/commands/
cat > .helix/commands/test.md << 'EOF'
---
name: test-cmd
description: Test command
---
Test prompt
EOF
helix commands list | grep test-cmd && echo "PASS: Custom
        Commands" || echo "FAIL: Custom Commands"

# Test 6: Checkpoint
helix checkpoint save --name "test-checkpoint"
helix checkpoint list | grep test-checkpoint && echo "PASS:
        Checkpoint" || echo "FAIL: Checkpoint"

# ---- Core Features ----
echo "--- Core Features ---"

# Test 7: SKILL.md Discovery
mkdir -p ~/.helix/skills/test-skill/
cat > ~/.helix/skills/test-skill/SKILL.md << 'EOF'
---
name: test-skill
description: A test skill
---
# Instructions
Test instructions here.
EOF
helix plugin list | grep test-skill && echo "PASS: SKILL.md" ||
        echo "FAIL: SKILL.md"

# Test 8: Plan Mode
# (Requires interactive test)

# Test 9: Command Validation
helix shell validate "ls -la" && echo "PASS: Valid command" ||
        echo "FAIL: Valid command"
```

```
helix shell validate "rm -rf /" && echo "FAIL: Invalid command
        accepted" || echo "PASS: Invalid command rejected"

# Test 10: Recipe Execution
cat > /tmp/test-recipe.yml << 'EOF'
name: test-recipe
description: Test recipe
steps:
  - name: step1
    action: echo "Hello from recipe"
EOF
helix recipe run /tmp/test-recipe.yml && echo "PASS: Recipe" ||
        echo "FAIL: Recipe"

# Test 11: Worktree
cd /tmp && git init test-repo && cd test-repo && echo "init" >
        file.txt && git add . && git commit -m "init"
helix worktree create --agent "test"
git worktree list | grep "agent-" && echo "PASS: Worktree" ||
        echo "FAIL: Worktree"
helix worktree remove $(git worktree list | grep "agent-" | awk
        '{print $1}')

# Test 12: Shared Memory
helix memory shared write --key "test" --value "hello" --role
        test
result=$(helix memory shared read --key "test" --role test)
[[ "$result" == *"hello"* ]] && echo "PASS: Shared Memory" ||
        echo "FAIL: Shared Memory"

# ---- Advanced Features ----
echo "--- Advanced Features ---"

# Test 13: Agent Teams
helix team create --agents 2 --task "Parallel test"
# Verify agents file coordination

# Test 14: Python Sandbox
helix sandbox python "print(2+2)" | grep "4" && echo "PASS:
        Python Sandbox" || echo "FAIL: Python Sandbox"

# Test 15: Database Tools
# (Requires running postgres)
# helix db connect "postgres://localhost/test"
# helix db query "SELECT 1" | grep "1" && echo "PASS: Database"
        || echo "FAIL: Database"

# Test 16: Adversary Reviewer
```

```
helix security review --action "eval(user_input)" | grep "FAIL"
        && echo "PASS: Adversary" || echo "FAIL: Adversary"

# Test 17: Multi-model
helix llm switch --provider openai --model gpt-4o
helix chat --message "test"
# Verify correct provider used

# Test 18: Terminal Agent Mode
# (Requires TUI test framework)

echo ""
echo "=== TEST SUITE COMPLETE ==="
```

Integration Verification Checklist

For each feature, verify:

1. **Compilation**: `go build ./...` succeeds
2. **Unit Tests**: `go test ./internal/<package>/...` passes
3. **Integration Tests**: Feature works in isolation
4. **End-to-End**: Feature works in full workflow
5. **Documentation**: Feature documented in CLI help
6. **Config Migration**: Existing configs work with new features
7. **Security**: No new vulnerabilities introduced
8. **Performance**: No regressions in baseline benchmarks

Appendix A: Agent Priority Matrix

Agent	P0	P1	P2	Rationale
Claude-Code-Plugins	X			Skills ecosystem is industry direction
Codename_Goose	X			MCP deep integration, recipes, security
Emdash	X			Parallel agents, worktree, issue integration
GitHub-Copilot-CLI	X			Plan mode, custom agents - user expectations
Mistral_Code	X			Qwen Coder, modes, todo - high-value features
Qwen_Code	X			1M context, competitive model
TaskWeaver	X			Code-first, multi-agent roles - unique value
Warp	X			Terminal-native UX - paradigm shift
gptme		X		

Agent	P0	P1	P2	Rationale
				Self-programming, multi-model, extensible
Bridle		X		Cross-harness compatibility
GitHub-Spec-Kit		X		Spec management
GitMCP		X		Git operations via MCP
Nanocoder		X		Checkpoints, custom commands
Shai		X		Shell assistant, OpenAI-compatible server
Stark-Kitty-Kiro-Cli		X		Terminal adaptation, external editor
vtcode		X		Security sandboxing
Codai		X		Session-based CLI
Conduit			X	Visual workflows (desktop only)
DeepSeek_CLI			X	Provider already exists
FauxPilot			X	Local completion overlap
Gemini_CLI			X	Provider already exists
Get-Shit-Done			X	Simple focus mode
MobileAgent			X	Mobile-specific
Multiagent-Coding			X	Pattern library
Noi			X	Desktop app (Electron)
Octogen			X	AutoGen patterns
Ollama_Code			X	Ollama already exists
Postgres-MCP			X	Database tools
SnowCLI			X	Snowflake-specific
Superset			X	BI-specific

Appendix B: New Files Summary

#	File Path	Purpose	Lines (est)
1	`internal/skills/registry.go`	SKILL.md progressive disclosure	200
2	`internal/skills/loader.go`	Skill loading from YAML frontmatter	150
3	`internal/skills/watcher.go`	Live reload with fsnotify	100
4	`cmd/helix/plugin.go`	Plugin marketplace CLI	120

#	File Path	Purpose	Lines (est)
5	`cmd/helix/profile.go`	Configuration profiles CLI	100
6	`internal/config/harness/translator.go`	Cross-harness config translation	150
7	`internal/config/harness/matrix.go`	Path mapping between harnesses	100
8	`internal/mcp/apps/renderer.go`	Interactive UI in MCP responses	150
9	`internal/mcp/apps/ui.go`	Button/form rendering for TUI	120
10	`internal/workflow/recipes/recipe.go`	Portable workflow definitions	120
11	`internal/workflow/recipes/executor.go`	Recipe execution engine	100
12	`internal/agent/security/adversary.go`	Security review agent	120
13	`internal/git/worktree.go`	Git worktree management	180
14	`internal/integrations/issues/connector.go`	Unified issue tracker interface	100
15	`internal/integrations/issues/github.go`	GitHub issue connector	150
16	`internal/integrations/issues/jira.go`	Jira connector	150
17	`internal/integrations/issues/linear.go`	Linear connector	120
18	`internal/session/modes.go`	Plan/execute/architect mode controller	130
19	`internal/agent/custom/loader.go`	Custom agent from markdown	120
20	`internal/tools/todo.go`	Session todo management	150
21	`internal/memory/checkpoint.go`	Conversation snapshots	120
22	`internal/commands/custom.go`		140

#	File Path	Purpose	Lines (est)
		User-defined commands from markdown	
23	`internal/commands/context.go`	/context slash commands	100
24	`internal/editor/external.go`	External editor launch	80
25	`internal/tools/sandbox/python.go`	Python code execution	180
26	`internal/memory/shared.go`	Cross-role shared memory	120
27	`internal/agent/types/planner_agent.go`	Task decomposition planner	100
28	`internal/agent/types/code_interpreter.go`	Code execution agent	80
29	`internal/agent/types/code_generator.go`	Code creation agent	80
30	`internal/agent/types/code_executor.go`	Safe execution agent	80
31	`internal/tools/shell/security.go`	Per-command validation	150
32	`internal/sandbox/landlock_linux.go`	Linux Landlock sandboxing	100
33	`internal/sandbox/seatbelt_darwin.go`	macOS Seatbelt sandboxing	100
34	`internal/agent/self_modify.go`	Agent self-modification	100
35	`internal/tools/superset/client.go`	Superset API client	120
36	`internal/tools/database/snowflake.go`	Snowflake operations	150
37	`internal/tools/database/postgres.go`	Postgres operations	200
38	`internal/mcp/tools/git.go`	Git MCP tool definitions	150
39	`internal/mcp/tools/github.go`	GitHub MCP tool definitions	150

#	File Path	Purpose	Lines (est)
40	`internal/session/focus.go`	Pomodoro focus mode	80
41	`internal/memory/team/library.go`	Team knowledge library	100
42	`internal/memory/team/search.go`	Semantic search over team knowledge	80
43	`internal/tools/sandbox/dataframe.go`	DataFrame serialization	100
44	`internal/llm/parsing/nested.go`	Multi-level tool call parsing	100
45	`internal/llm/parsing/model_specific.go`	Model-specific output parsers	80
46	`internal/memory/prompts.go`	Prompt library and templates	80
47	`internal/context/providers/shai_provider.go`	SHAI.md context provider	60
48	`internal/workflow/triggers/event_trigger.go`	Event-based workflow triggers	80
49	`internal/workflow/triggers/schedule_trigger.go`	Scheduled workflow triggers	60
50	`internal/workflow/blocks/transform.go`	Data transformation blocks	80
51	`internal/agent/swarm/patterns.go`	Agent orchestration patterns	120
52	`internal/agent/comm/messaging.go`	Inter-agent messaging	100
53	`internal/agent/comm/routing.go`	Message routing between agents	80
54	`internal/agent/verifier/cross_agent.go`	Cross-agent validation	80
55	`internal/specs/spec.go`	Specification document management	80
56	`internal/llm/local/fauxpilot.go`	FauxPilot local completion	100
57	`internal/tools/mobile/adb.go`	ADB device control	120

#	File Path	Purpose	Lines (est)
58	`internal/llm/vision/ui_analysis.go`	UI element detection from screenshots	80
59	`internal/session/yolo_mode.go`	Yolo execution mode	60
60	`cmd/server/openai_compat.go`	OpenAI-compatible API	120

Total New Files: 60 **Total Estimated Lines**: ~7,500

Appendix C: Modified Files Summary

#	File Path	Changes
1	`internal/agent/swarm/coordinator.go`	Add AgentTeam, SharedTaskList, MailboxSystem
2	`internal/llm/compression/auto_compact.go`	Add 95% threshold auto-trigger
3	`internal/context/builder.go`	Add full-project scanning
4	`internal/llm/streaming/`	Enhance SSE streaming for all providers
5	`internal/llm/vision/`	Extend with UI analysis
6	`applications/terminal_ui/terminal.go`	Add agent mode, progress indicators, hyperlinks
7	`applications/desktop/`	Add workflow canvas, agent kanban
8	`internal/workflow/`	Add recipe execution hooks
9	`internal/tools/shell/`	Add multiline support, failure detection
10	`api/openapi.yaml`	Add completion endpoints, IDE plugin endpoints
11	`cmd/server/main.go`	Add OpenAI-compatible routes
12	`internal/tools/web/`	Add web search capabilities
13	`internal/memory/`	Add prompt library, checkpoint hooks
14	`internal/session/manager.go`	Add mode controller integration
15	`internal/agent/types/`	Add new agent type constants
16	`internal/context/providers/`	Add SHAI.md, instructions.md providers
17	`internal/llm/factory.go`	Add fauxpilot, enhanced parsers
18	`cmd/helix/root.go`	

#	File Path	Changes
		Add plugin, profile, recipe subcommands
19	`internal/commands/builtin/`	Add condense, context management
20	`internal/config/`	Add harness translation, profile migration

Total Modified Files: 20

Appendix D: Dependencies to Add

```
// go.mod additions
github.com/lib/pq v1.10.9           // Postgres
github.com/fsnotify/fsnotify v1.7.0  // File watching
sigs.k8s.io/yaml v1.4.0             // YAML parsing
github.com/snowflakedb/gosnowflake v1.7.0 // Snowflake
```

Document generated for HelixCode integration. All agents analyzed from helix_agent/cli_agents submodule references. For each feature, the Anti-Bluff Test provides an objective verification that the port is functional end-to-end.